

ALTHEA · MEDICAL · SYSTEMS

**althea.**

PLATEFORME E-COMMERCE DE MATÉRIEL MÉDICAL

DOSSIER DE CONCEPTION TECHNIQUE — DCT

# § Architecture, sécurité et déploiement d'une plateforme e-commerce médicale

*Document technique détaillé — frontend, backend, infrastructure, conformité.*

## PROJET

**Althea Systems**  
Bachelor 3 — Coordinateur de  
Projets Informatiques  
RNCP 34581 · Sup de Vinci

## VERSION &amp; DATE

**v2.0 — 7 mai 2026**  
Livrable VI.5 — Documentation  
finale

## AUTEURS

## REPOSITORY

Samy Abdelmalek (*chef de projet*)  
Kelvin Chauvel · Tristan San-Juan ·  
Rayane Menkar

[github.com/itsaam/althea-](https://github.com/itsaam/althea-systems)  
[systems](#)  
Production · [althea.vjuya.me](https://althea.vjuya.me)

# Dossier de Conception Technique — Althea Systems

Version 2.0 — 07 mai 2026 Plateforme e-commerce B2B/B2C de matériel médical

## § Préambule

Ce document est le **Dossier de Conception Technique (DCT)** de la plateforme **Althea Systems**. Il s'inscrit dans le cadre du livrable final du projet fil rouge Bachelor 3 — **Coordinateur de Projets Informatiques** (RNCP 34581) — Sup de Vinci.

### Objectif

Permettre à une équipe IT externe de **comprendre, maintenir, faire évoluer** et **reproduire** la plateforme Althea Systems à partir des seules informations contenues dans ce document et du code source versionné.

Le DCT couvre les six axes imposés par le cadre pédagogique :

1. **Architecture du système** — schémas, technologies, choix structurants
2. **Modèle de données** — structure de la base, diagramme ERD, énumérations
3. **Fonctionnalités implémentées** — documentation précise et cas d'usage
4. **Structure du code** — organisation, conventions, gestion des erreurs
5. **Sécurité** — authentification, validation, headers, RGPD, PCI-DSS
6. **Déploiement** — environnements, procédures, versionning, CI/CD

### Périmètre

Le présent document détaille **l'intégralité de l'application**, frontend et backend, ainsi que toute l'infrastructure de production. Il ne couvre ni le détail du document de cadrage (livrable VI.3 distinct), ni la dynamique d'équipe (livrable VI.6 individuel).

### Conventions d'écriture

- Les blocs de code et noms techniques sont en **monospace**.
- Les chemins de fichier utilisent la racine projet : `src/...`, `prisma/...`, `docker/...`.
- Les diagrammes utilisent la syntaxe **Mermaid** (rendus dans le PDF).
- Les références internes sont des liens cliquables dans le PDF généré.

Équipe projet et répartition

| MEMBRE           | RÔLE PRINCIPAL                  | PÉRIMÈTRE TECHNIQUE                                                                                                                              |
|------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Samy Abdelmalek  | Chef de projet, lead full-stack | Architecture globale, frontend (design system, i18n, SEO, animations), backend (auth, paiement, admin, chatbot), DevOps (Docker, Dokploy, CI/CD) |
| Kelvin Chauvel   | Développeur backend             | API produits, gestion catalogue, validation Zod                                                                                                  |
| Tristan San-Juan | Développeur full-stack          | Composants UI, intégration Stripe, formulaires                                                                                                   |
| Rayane Menkar    | Développeur frontend            | Pages publiques, composants partagés                                                                                                             |

Répartition des contributions selon l'historique Git (au 07 mai 2026) — multiples pseudos GitHub utilisés : Samy Abdelmalek (alias `vjuya`, `itsaam`, `May`) — 255 commits ; Kelvin Chauvel — 46 commits ; Tristan San-Juan (`Mitikx`) — 27 commits ; Rayane Menkar — 8 commits.

## § Table des matières

1. [Présentation du projet](#)
  1. Contexte métier
  2. Objectifs fonctionnels
  3. Cibles et utilisateurs
  4. Périmètre fonctionnel
2. [Architecture du système](#)
  1. Vue d'ensemble
  2. Stack technique complète
  3. Architecture en couches
  4. Diagramme d'architecture globale
  5. Flux de données
  6. Choix architecturaux et justifications
3. [Modèle de données](#)
  1. Vue d'ensemble du schéma
  2. Diagramme ERD complet
  3. Description détaillée des entités
  4. Énumérations
  5. Relations et contraintes
  6. Migrations Prisma
4. [Fonctionnalités implémentées](#)
  1. Vitrine publique
  2. Espace client
  3. Tunnel de commande
  4. Back-office d'administration
  5. Authentification
  6. Internationalisation
  7. Chatbot IA
  8. Emails transactionnels
  9. SEO
  10. Cas d'usage détaillés
5. [Structure du code](#)
  1. Arborescence du projet

2. Organisation des dossiers
3. Conventions de nommage
4. Couche de services
5. Gestion des erreurs
6. Validation des entrées
7. Logging structuré
8. Tests

## 6. [Sécurité](#)

1. Modèle de menaces
2. Authentification
3. Autorisation
4. Validation et sanitization
5. Headers de sécurité
6. Rate limiting
7. Sécurisation des paiements
8. RGPD
9. Audits de sécurité

## 7. [Déploiement](#)

1. Architecture de déploiement
2. Conteneurisation Docker
3. Pipeline CI/CD
4. Variables d'environnement
5. Procédure de déploiement
6. Rollback
7. Backups
8. Monitoring
9. Versionning

## 8. [Annexes](#)

1. Catalogue des 72 endpoints API
  2. Catalogue des composants
  3. Variables d'environnement complètes
  4. Glossaire
  5. Ressources et références
-



# 1. Présentation du projet

## § 1.1 Contexte métier

Althea Systems est une plateforme e-commerce dédiée à la **distribution de matériel médical professionnel** (consommables, instruments, équipements) pour cabinets médicaux, cliniques, pharmacies et établissements de santé.

Le marché du matériel médical en France représente environ **30 milliards d'euros annuels** (source : SNITEM, 2024) et reste majoritairement adressé par des canaux de vente traditionnels (commerciaux, catalogues papier, fax). La digitalisation du processus d'achat est un enjeu structurant pour les établissements de santé qui cherchent à :

- Réduire les coûts d'approvisionnement (négociation, comparaison)
- Accélérer le processus commande → livraison
- Tracer les achats pour la traçabilité réglementaire
- Centraliser la gestion multi-sites

Althea Systems répond à ce besoin en proposant un **catalogue digital structuré**, une **gestion de comptes professionnels** (TVA, factures conformes), un **tunnel d'achat sécurisé** et une **interface d'administration** pour le distributeur.

## § 1.2 Objectifs fonctionnels

| OBJECTIF                | DESCRIPTION                                                                                                                                                  | CRITÈRE DE SUCCÈS                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Catalogue produit riche | Présenter les références médicales avec spécifications techniques, images haute résolution, classification fiscale (TVA 0/5,5/10/20%), informations de stock | 100% des produits publiés disposent d'au moins une image, un prix HT et une catégorie |
| Recherche performante   | Permettre la recherche full-text avec filtres (catégorie, prix, disponibilité, TVA) et résultats sous 200 ms (P95)                                           | < 200 ms sur 10 000 produits                                                          |
| Tunnel d'achat sécurisé | Paiement carte bancaire conforme PCI-DSS, gestion d'adresses multiples, calcul automatique de TVA et frais de port                                           | 0 faille critique sur l'audit sécurité paiement                                       |
| Compte professionnel    | Historique des commandes, téléchargement des factures PDF, gestion des moyens de paiement enregistrés, export RGPD                                           | Conformité RGPD complète (Art. 13, 15, 16, 17, 20, 21)                                |
| Back-office complet     | CRUD sur produits, catégories, commandes, utilisateurs, carrousel, messages contact, avoirs ; tableau de bord avec KPI                                       | Toutes les opérations métier réalisables sans accès direct à la base                  |



| OBJECTIF                            | DESCRIPTION                                                                                                    | CRITÈRE DE SUCCÈS                  |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>Conformité fiscale française</b> | Génération de factures avec mentions légales obligatoires (SIRET, TVA intracommunautaire, conditions de vente) | 100% des factures conformes au CGI |

## § 1.3 Cibles et utilisateurs

L'application sert quatre profils distincts, chacun avec son parcours et ses droits :

| PROFIL                  | AUTHENTIFICATION                                 | PÉRIMÈTRE                                                                                       |
|-------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Visiteur anonyme</b> | Aucune                                           | Catalogue, recherche, fiches produit, panier (cookie), pages légales, formulaire de contact     |
| <b>Client B2C</b>       | Email/mot de passe ou OAuth (Google, GitHub)     | Vitrine + commande, profil, adresses, historique commandes, factures                            |
| <b>Client B2B</b>       | Email/mot de passe ou OAuth                      | Mêmes droits que B2C + numéro TVA intracommunautaire sur les factures                           |
| <b>Administrateur</b>   | Email/mot de passe + <b>2FA TOTP obligatoire</b> | Back-office complet ( /admin/* ), gestion catalogue, commandes, utilisateurs, contenu éditorial |

## § 1.4 Périmètre fonctionnel

Le périmètre couvre quatre grands domaines :

### Vitrine et catalogue

- Page d'accueil éditoriale avec hero animé, carrousel produits dynamique, sections catégorisées
- Catalogue produits paginé avec filtres avancés (catégorie, prix, statut, TVA)
- Fiche produit avec galerie multi-images, spécifications techniques, calcul TVA, produits similaires
- Recherche full-text avec auto-complétion et résultats facetés
- Pages légales (CGU, mentions légales, politique de confidentialité, RGPD)
- Page contact avec formulaire RGPD

### Compte client

- Inscription avec vérification d'email
- Connexion email/mot de passe ou OAuth (Google, GitHub)
- Réinitialisation de mot de passe
- Profil utilisateur (nom, prénom, téléphone, photo)
- Carnet d'adresses avec adresse par défaut
- Historique des commandes avec filtres par année et statut

- Téléchargement des factures et avoirs PDF
- Gestion des moyens de paiement enregistrés (Stripe)
- Export RGPD des données personnelles
- Suppression de compte (droit à l'oubli)

## Tunnel d'achat

- Panier persistant (cookie côté client + base si connecté)
- Saisie d'adresse de livraison avec autocomplete API BAN (Base Adresse Nationale)
- Calcul automatique du sous-total, TVA (multi-taux), frais de port
- Paiement Stripe (Checkout hébergé ou carte enregistrée)
- Webhook Stripe sécurisé (signature vérifiée)
- Confirmation par email + facture PDF jointe
- Gestion des statuts de commande (PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED)
- Gestion des avoirs (CreditNote) en cas d'annulation ou de remboursement

## Back-office d'administration

- Dashboard avec KPIs (CA, commandes, clients, panier moyen, conversion)
  - Graphiques de ventes (7j / 30j / 90j / 1 an)
  - Répartition CA par catégorie
  - Analyse des paniers abandonnés
  - Alertes de stock bas
  - CRUD produits avec éditeur WYSIWYG (Tiptap), upload d'images vers R2, export Excel
  - CRUD catégories avec arborescence et drag-and-drop
  - Gestion des commandes (changement de statut, génération facture/avoir, timeline)
  - CRUD utilisateurs avec gestion des rôles
  - CRUD carrousel d'accueil avec WYSIWYG
  - Boîte de réception des messages contact avec réponse intégrée
  - Gestion des avoirs (CreditNote)
  - Console chatbot (visualisation et reprise de conversations)
-



## 2. Architecture du système

### § 2.1 Vue d'ensemble

Althea Systems repose sur une **architecture monolithique modulaire** construite autour de **Next.js 16 App Router**, qui colocalise le rendu serveur (SSR/RSC), les API REST et les composants React dans un même runtime Node.js 20.

Cette approche a été retenue pour les raisons suivantes :

- **Cohérence du typage** entre frontend et backend (TypeScript de bout en bout, schémas Zod partagés)
- **Performance** grâce au React Server Components qui élimine le waterfall fetch initial
- **Simplicité opérationnelle** d'un seul service à déployer, une seule base de code à maintenir
- **Isolation logique** par dossier ( `(site)` , `(account)` , `admin` , `api` ) qui préserve la lisibilité

Le service Next.js est complété par trois dépendances d'infrastructure :

- **PostgreSQL 16** comme base de données principale (relationnel, ACID, JSON natif)
- **Redis 7** pour le cache, le rate limiting, les sessions volatiles et les secrets temporaires (2FA setup)
- **Cloudflare R2** comme stockage objet S3-compatible pour les images produits et les PDF de factures

Et quatre services SaaS externes :

- **Stripe** pour l'encaissement carte bancaire et la gestion des moyens de paiement
- **Resend** pour les emails transactionnels (confirmation, vérification, mot de passe oublié, facture)
- **Google OAuth 2.0** et **GitHub OAuth** pour l'authentification fédérée
- **OpenAI API** pour le chatbot d'assistance client (modèle `gpt-4o-mini` en streaming)

### § 2.2 Stack technique complète

#### Couche applicative

| COMPOSANT | TECHNOLOGIE | VERSION | RÔLE                                                 |
|-----------|-------------|---------|------------------------------------------------------|
| Framework | Next.js     | 16.0.7  | App Router, SSR/RSC, API Routes, optimisation images |
| Runtime   | Node.js     | 20 LTS  | Exécution serveur                                    |
| Langage   | TypeScript  | 5.x     | Typage statique, contrats partagés frontend/backend  |

| COMPOSANT   | TECHNOLOGIE | VERSION           | RÔLE                                       |
|-------------|-------------|-------------------|--------------------------------------------|
| Compilateur | Turbopack   | inclus Next.js 16 | Bundler dev (Rust)                         |
| UI          | React       | 19.2              | Composants, Server Components, transitions |

Couche frontend

| COMPOSANT           | TECHNOLOGIE          | VERSION  | RÔLE                                                           |
|---------------------|----------------------|----------|----------------------------------------------------------------|
| Styling             | Tailwind CSS         | 4.x      | Utility-first CSS, configuration <code>@theme</code> inline    |
| Composants          | shadcn/ui + Radix UI | dernière | Composants accessibles (Dialog, Dropdown, Select, etc.)        |
| Animations          | Framer Motion        | 12.38    | Animations React déclaratives                                  |
| Animations avancées | GSAP + ScrollTrigger | 3.13     | Carrousel pin scroll, parallax                                 |
| Smooth scroll       | Lenis                | 1.3      | Défilement fluide global                                       |
| Formulaires         | React Hook Form      | 7.71     | Gestion formulaires performante                                |
| Validation client   | Zod                  | 4.1      | Schémas partagés avec le backend                               |
| State global        | Zustand              | 5.0      | Stores <code>auth</code> , <code>cart</code> , <code>ui</code> |
| Tables              | TanStack Table       | 8.21     | Tableaux admin avec tri/filtre/pagination                      |
| Drag-and-drop       | dnd-kit              | 6.3      | Réordonnancement carrousel et catégories                       |
| Carrousels          | Embla Carousel       | 8.6      | Carrousels produits accessibles                                |
| Éditeur WYSIWYG     | Tiptap               | 3.15     | Édition descriptions produits, sous-titres carrousel           |
| Charts              | Recharts             | 3.6      | Graphiques dashboard admin                                     |
| Icônes              | Lucide React         | 0.555    | Bibliothèque d'icônes SVG                                      |
| QR Code             | qrcode.react         | 4.2      | Génération QR pour 2FA TOTP                                    |
| Notifications       | Sonner               | 2.0      | Toasts                                                         |
| i18n                | next-intl            | 4.5      | Internationalisation FR/EN/AR avec RTL                         |

Couche backend

| COMPOSANT        | TECHNOLOGIE | VERSION | RÔLE                                              |
|------------------|-------------|---------|---------------------------------------------------|
| ORM              | Prisma      | 6.19    | Type-safe queries, migrations versionnées         |
| Authentification | NextAuth.js | 4.24    | OAuth (Google, GitHub), Credentials, JWT sessions |
| 2FA              | otplib      | 12.0    | TOTP (Time-based One-Time Password)               |

| COMPOSANT            | TECHNOLOGIE         | VERSION | RÔLE                                            |
|----------------------|---------------------|---------|-------------------------------------------------|
| Paement              | Stripe SDK          | 20.1    | Checkout, PaymentIntents, webhooks              |
| Email                | Resend SDK          | 6.5     | Templates React email, deliverability optimisée |
| Génération PDF       | @react-pdf/renderer | 4.3     | Factures, avoirs                                |
| IA conversationnelle | OpenAI SDK          | 6.34    | Streaming GPT-4o-mini                           |
| Hash mot de passe    | bcryptjs            | 3.0     | Hash bcrypt salt rounds = 10                    |
| Sanitization HTML    | DOMPurify           | 3.3     | Anti-XSS sur contenus WYSIWYG                   |
| Logging              | Winston             | 3.18    | Logging structuré (niveaux, sections)           |
| Validation           | Zod                 | 4.1     | Schémas de validation runtime                   |
| HTTP client R2       | @aws-sdk/client-s3  | 3.946   | S3 SDK (compatible R2)                          |
| Export Excel         | ExcelJS             | 4.4     | Export XLSX (alternative sécurisée à xlsx)      |
| Driver PostgreSQL    | pg                  | 8.16    | Client PG natif                                 |
| Cache                | ioredis             | 5.8     | Client Redis avec lazy initialization           |

Couche infrastructure

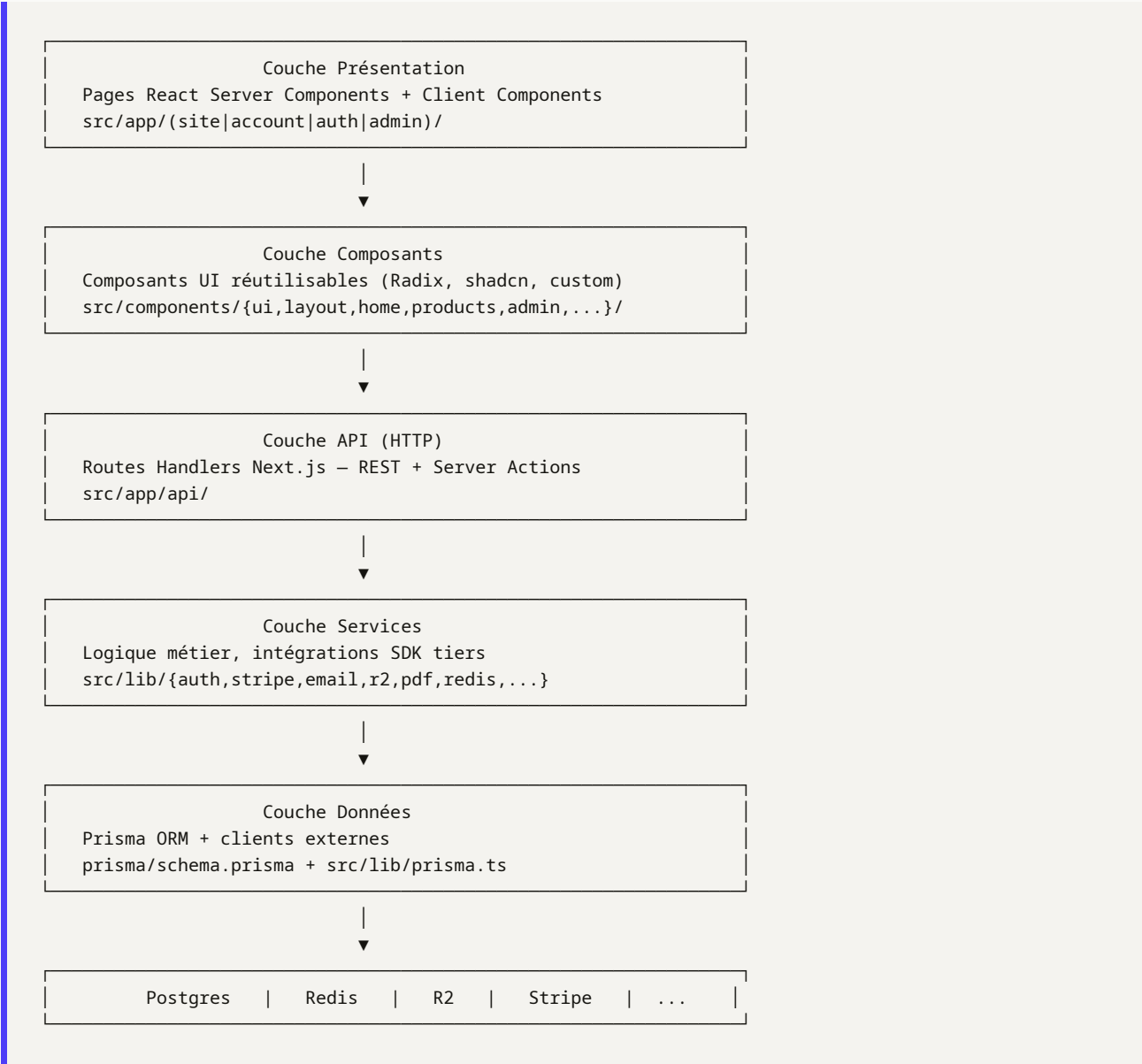
| COMPOSANT        | TECHNOLOGIE    | VERSION        | RÔLE                                        |
|------------------|----------------|----------------|---------------------------------------------|
| Conteneurisation | Docker         | 24+            | Build multi-stage, images Alpine légères    |
| Orchestration    | Docker Compose | v2             | Stack locale (app + Postgres + Redis)       |
| Hébergement      | Dokploy        | dernière       | PaaS auto-hébergé (auto-deploy, monitoring) |
| Reverse proxy    | Traefik        | inclus Dokploy | TLS Let's Encrypt automatique               |
| Base de données  | PostgreSQL     | 16 Alpine      | Stockage relationnel principal              |
| Cache            | Redis          | 7 Alpine       | Cache, rate limiting, sessions              |
| Stockage objet   | Cloudflare R2  | —              | S3-compatible, 0 € egress, CDN mondial      |
| CDN              | Cloudflare     | —              | DNS, proxy, mise en cache statique          |
| Monitoring       | UptimeRobot    | gratuit        | Surveillance HTTP toutes les 5 min          |
| CI/CD            | GitHub Actions | —              | Lint, typecheck, build, security audit      |
| Hébergement code | GitHub         | —              | Repo <a href="#">itsaam/althea-systems</a>  |

Couche tests

| COMPOSANT         | TECHNOLOGIE            | VERSION | RÔLE                                 |
|-------------------|------------------------|---------|--------------------------------------|
| Tests unitaires   | Vitest                 | 4.0     | Tests rapides en Node + jsdom        |
| Tests intégration | Vitest                 | 4.0     | Endpoints API avec base de test      |
| Tests E2E         | Playwright             | 1.59    | Tests bout en bout multi-navigateurs |
| Testing Library   | @testing-library/react | 16.3    | Tests composants centrés utilisateur |

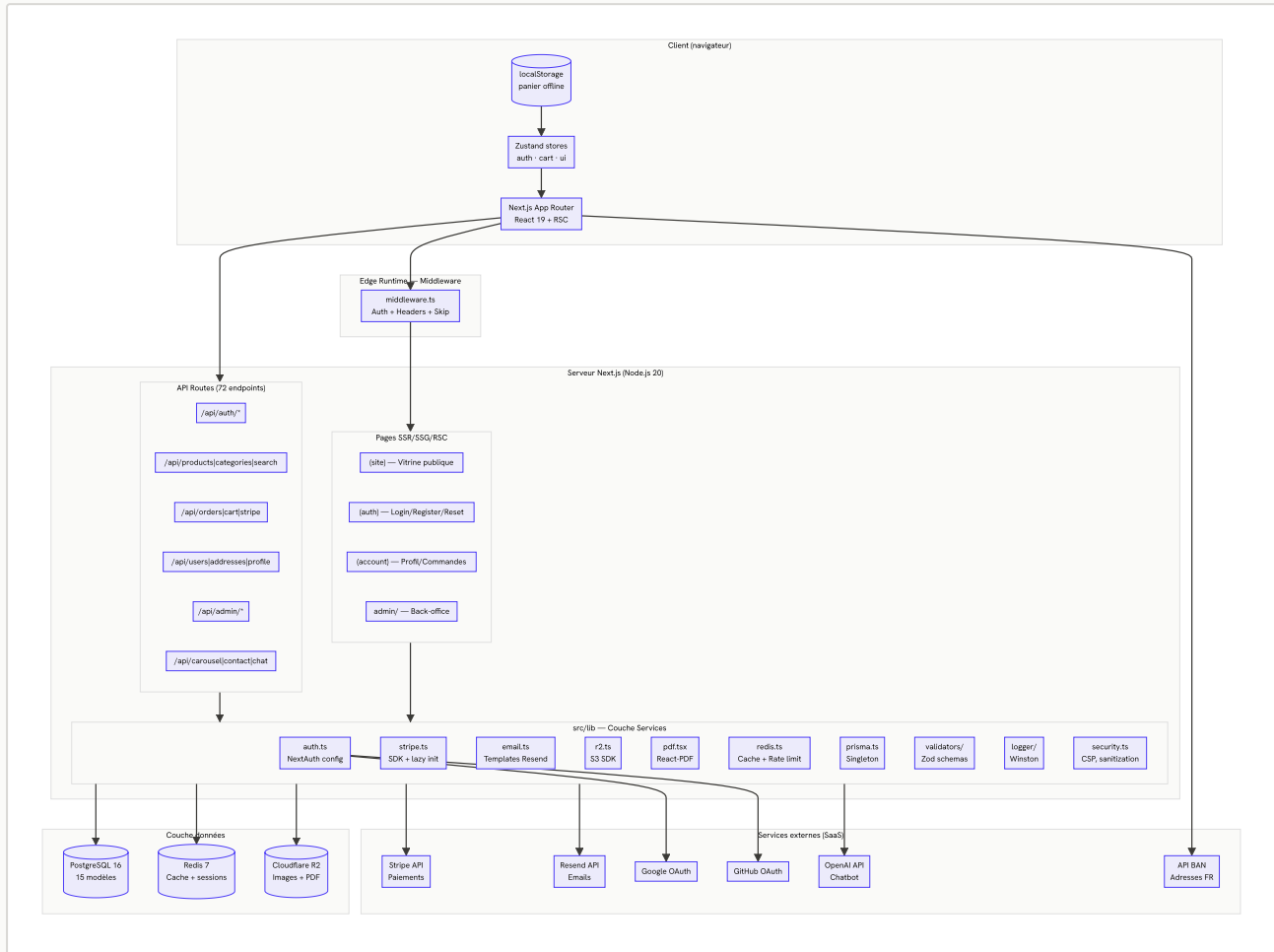
§ 2.3 Architecture en couches

L'application suit une **architecture en couches** strictement séparées :



Chaque couche n'appelle que la couche immédiatement inférieure. Les composants ne touchent jamais directement à Prisma ; les routes API ne contiennent jamais de logique de présentation.

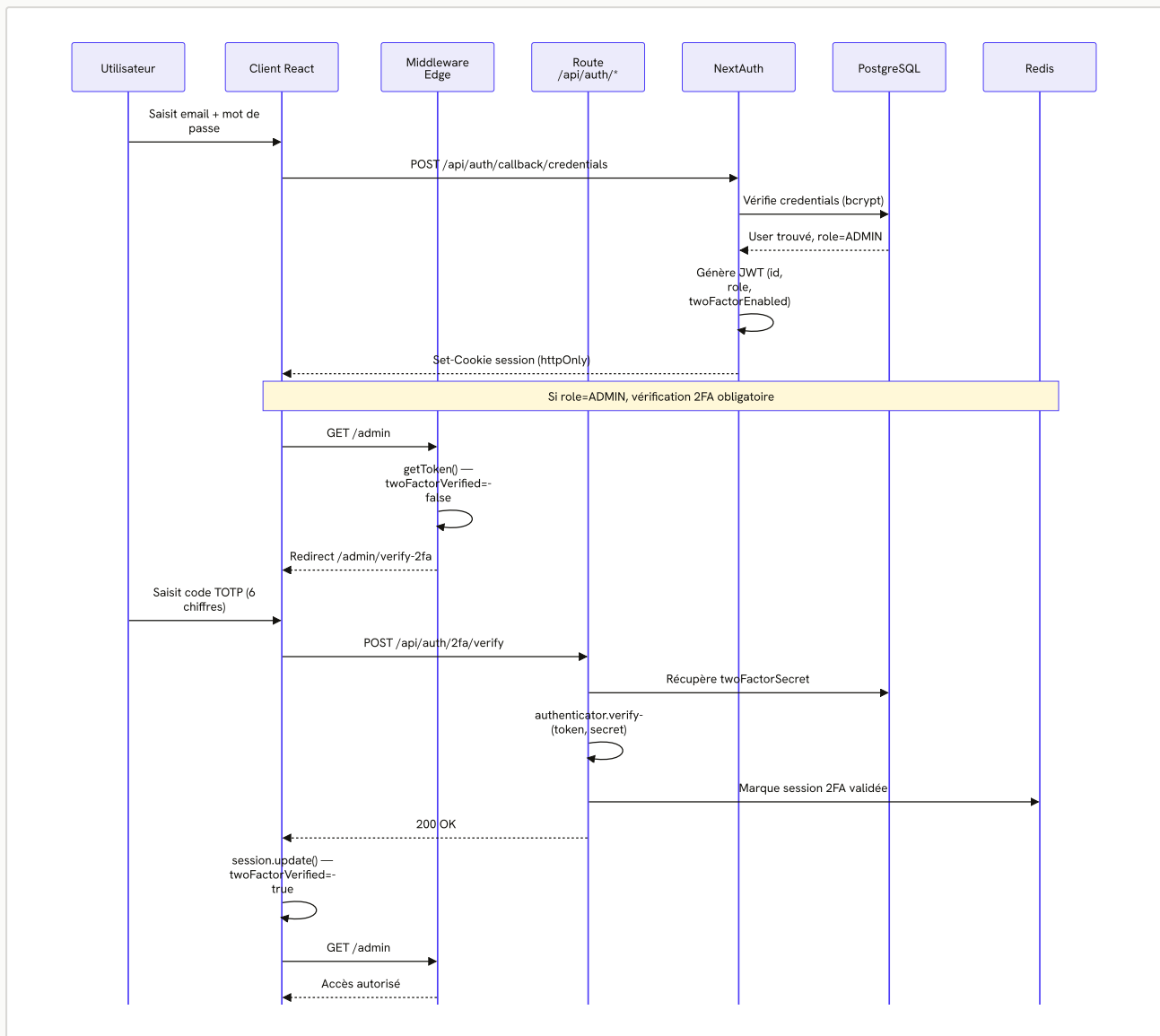
## § 2.4 Diagramme d'architecture globale



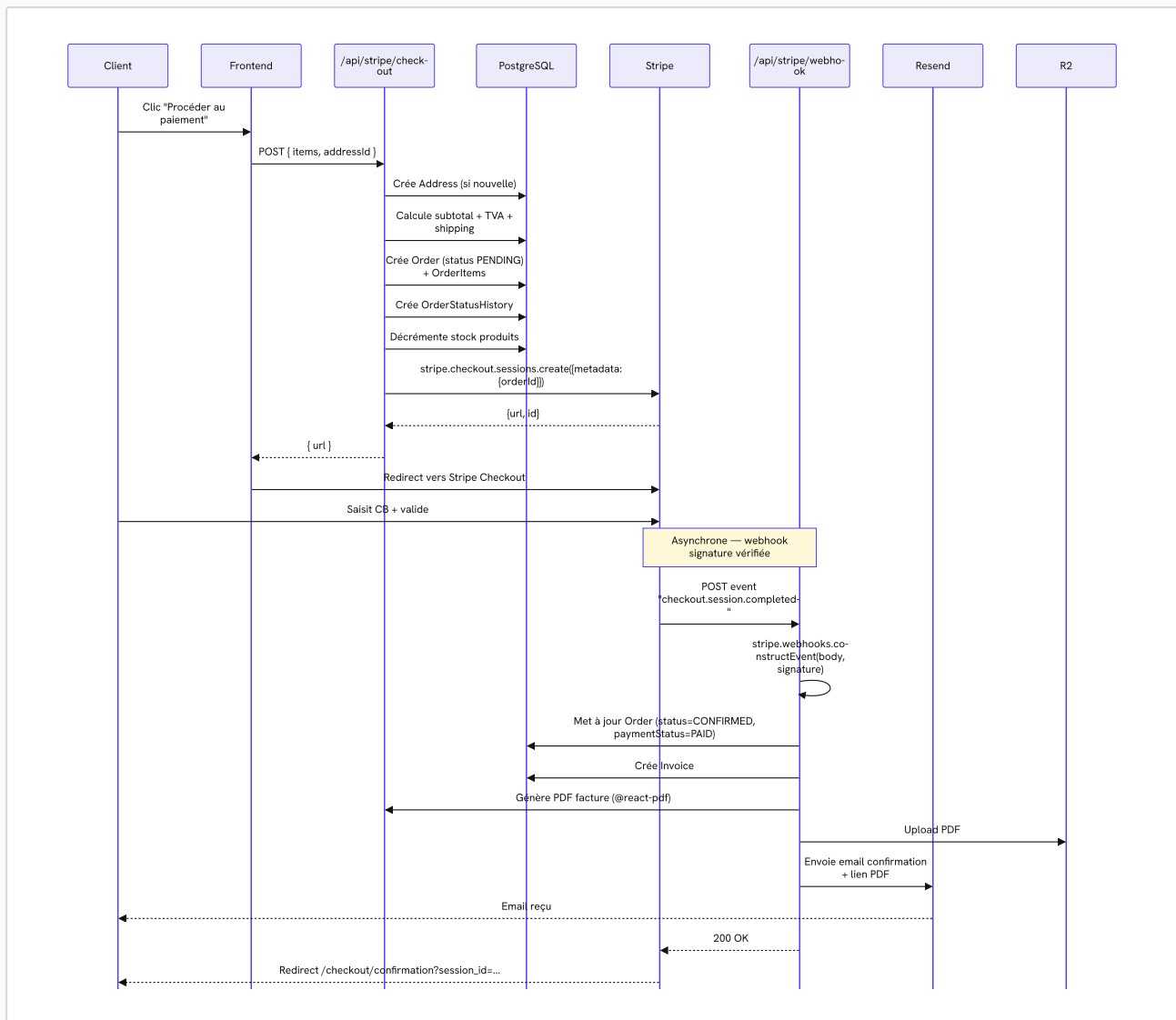


## § 2.5 Flux de données principaux

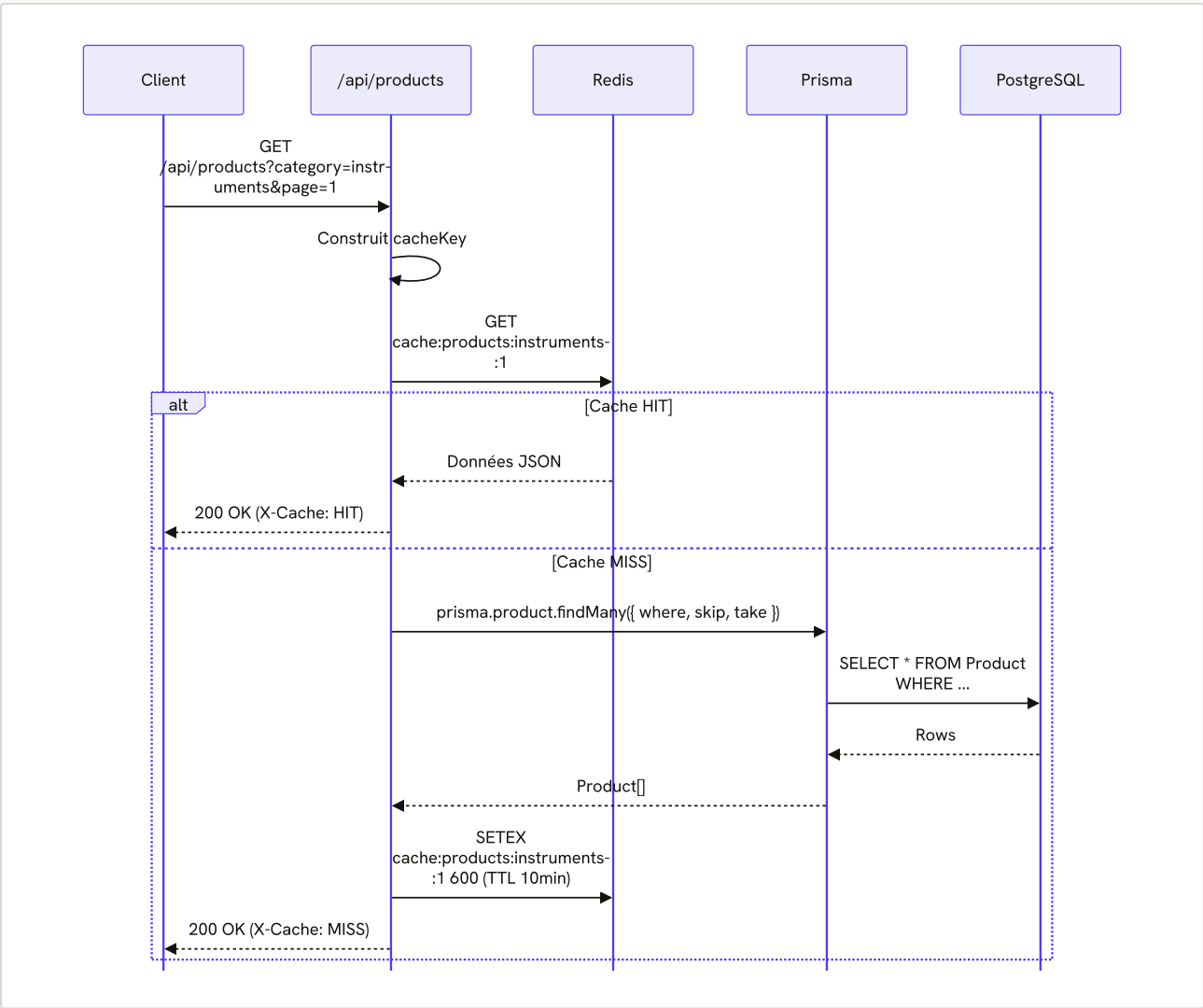
### 2.5.1 Flux authentification (credentials + 2FA)



## 2.5.2 Flux paiement (checkout Stripe)



### 2.5.3 Flux lecture catalogue (avec cache)



## § 2.6 Choix architecturaux et justifications

### 2.6.1 Pourquoi Next.js 16 (App Router) ?

| CRITÈRE                     | JUSTIFICATION                                                                  |
|-----------------------------|--------------------------------------------------------------------------------|
| Colocation frontend/backend | Un seul service, un seul typage TypeScript de bout en bout                     |
| React Server Components     | Élimination des waterfalls fetch, réduction du JavaScript envoyé au client     |
| Streaming SSR               | Affichage progressif (Suspense), TTFB optimisé                                 |
| API Routes intégrées        | Pas de backend séparé Express/Fastify à maintenir                              |
| Optimisation images         | <code>next/image</code> avec resize, format moderne (WebP, AVIF), lazy loading |
| Middleware Edge             | Auth et headers exécutés en bordure, latence minimale                          |

| CRITÈRE   | JUSTIFICATION                            |
|-----------|------------------------------------------|
| Turbopack | Builds incrémentaux ultra-rapides en dev |

Alternatives écartées : Remix (moins mature), SvelteKit (équipe React), Nuxt (équipe React).

### 2.6.2 Pourquoi PostgreSQL et pas MongoDB ?

Le cahier des charges initial mentionnait MongoDB. Après analyse technique, le choix s'est porté sur PostgreSQL :

| CRITÈRE                 | POSTGRESQL                                         | MONGODB                          |
|-------------------------|----------------------------------------------------|----------------------------------|
| Modèle                  | Relationnel (commandes, items, factures, adresses) | Documents                        |
| ACID                    | Oui (transactions multi-tables)                    | Oui mais limité multi-documents  |
| Jointures               | Performantes (Order → User → Address)              | Coûteuses (\$lookup)             |
| TVA, calculs financiers | Type Decimal natif précis                          | Float = imprécision              |
| Maturité ORM            | Prisma excellent                                   | Prisma moins puissant côté Mongo |
| Coût                    | Container léger                                    | Atlas payant à grande échelle    |

Le modèle de données d'Althea est **fondamentalement relationnel** : un User a N Adresses, une Order a 1 Address et N OrderItems, chaque OrderItem référence un Product. PostgreSQL est l'outil naturel pour cela.

MongoDB a été retiré du projet en décembre 2025 (commit 0319d87 ).

### 2.6.3 Pourquoi Cloudflare R2 et pas du base64 en BDD ?

Stocker les images en base64 dans MongoDB ou PostgreSQL multiplie la taille de stockage par 1,33 (overhead encodage), ralentit toutes les requêtes (chargement de blobs), empêche le CDN, et fait exploser les backups.

R2 est **S3-compatible, gratuit jusqu'à 10 Go, 0 € egress** (contrairement à AWS S3), avec CDN Cloudflare intégré (latence < 50 ms mondiale).

PostgreSQL stocke uniquement l'URL R2 ; R2 stocke le binaire ; le CDN sert l'image.

### 2.6.4 Pourquoi Redis et pas un cache mémoire Node ?

| CRITÈRE                  | REDIS                    | CACHE IN-MEMORY |
|--------------------------|--------------------------|-----------------|
| Partage entre instances  | Oui (scaling horizontal) | Non             |
| Persistence redémarrages | Oui (AOF)                | Non             |

| CRITÈRE                        | REDIS                              | CACHE IN-MEMORY |
|--------------------------------|------------------------------------|-----------------|
| TTL natif                      | Oui ( <code>SETEX</code> )         | À implémenter   |
| Rate limiting distribué        | Oui ( <code>INCR + EXPIRE</code> ) | Non             |
| Sessions volatiles (2FA setup) | Oui                                | Perte au reload |

Redis est utilisé pour 4 cas d'usage : cache catalogue (TTL 10 min), cache catégories (30 min), rate limiting par IP, secrets temporaires 2FA (TTL 10 min).

### 2.6.5 Pourquoi NextAuth.js v4 ?

| CRITÈRE                    | JUSTIFICATION                                                                                         |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| Intégration native Next.js | Configuration unique <code>src/lib/auth.ts</code>                                                     |
| Multi-providers            | Credentials, Google OAuth, GitHub OAuth — extensible                                                  |
| Sessions JWT stateless     | Pas de requête BDD pour valider chaque requête                                                        |
| Edge-runtime compatible    | Le middleware peut décoder le JWT sans accès BDD                                                      |
| Adapter Prisma             | Persiste <code>Account</code> , <code>Session</code> , <code>VerificationToken</code> automatiquement |
| Communauté                 | Maintenu par Vercel, documentation riche                                                              |

**Alternatives écartées :** Clerk (SaaS payant), Auth0 (SaaS payant), Auth.js v5 (encore en bêta au démarrage du projet).

### 2.6.6 Pourquoi Stripe (et pas PayPlug, Mollie, Adyen) ?

- PCI-DSS Level 1 automatique (les cartes ne touchent jamais notre serveur)
- Webhooks fiables avec signature HMAC vérifiable
- SDK TypeScript de qualité (typage exhaustif des objets Stripe)
- Multi-devises prêt à l'emploi
- Test mode complet avec cartes de test
- Documentation référence dans l'industrie
- Frais compétitifs : 1,4 % + 0,25 € par transaction CB européenne

### 2.6.7 Pourquoi Dokploy et pas Vercel ?

| CRITÈRE  | DOKPLOY                                         | VERCEL                               |
|----------|-------------------------------------------------|--------------------------------------|
| Coût     | VPS ~10 €/mois illimité                         | Free tier limité, payant à l'échelle |
| Contrôle | Total (Docker, Postgres, Redis sur le même VPS) | Limité (fonctions serverless)        |

| CRITÈRE         | DOKPLOY                          | VERCEL                                    |
|-----------------|----------------------------------|-------------------------------------------|
| Limite fonction | Aucune (Node.js classique)       | 10s sur Free, 60s sur Pro                 |
| Données         | Postgres + Redis sur le même VPS | Service tiers obligatoire (Neon, Upstash) |
| Vendor lock-in  | Aucun (stack 100 % open source)  | Fort                                      |
| Auto-deploy     | Push main → rebuild + redeploy   | Identique                                 |

Le seul inconvénient est l'absence de CDN edge mondial (mais Cloudflare devant le VPS comble ce manque).

2.6.8 Pourquoi Tailwind 4 et pas styled-components ?

| CRITÈRE             | TAILWIND 4                               | STYLED-COMPONENTS             |
|---------------------|------------------------------------------|-------------------------------|
| Performance runtime | Aucune (CSS statique compilé)            | Coût JS au runtime            |
| Bundle              | Purge auto (< 10 Ko en prod)             | Toutes les variantes incluses |
| DX                  | Auto-completion IDE, classes utilitaires | Composants stylés             |
| RSC compatible      | Oui                                      | Non (besoin de 'use client')  |
| Maintenance         | Pas de fichier CSS séparé à synchroniser | Risque de duplication         |
| Charte centralisée  | @theme inline dans globals.css           | ThemeProvider                 |

Tailwind 4 introduit @theme inline qui supprime le besoin de tailwind.config.ts — toutes les variables (couleurs, typographies, espacements) sont en CSS.



## 3. Modèle de données

### § 3.1 Vue d'ensemble du schéma

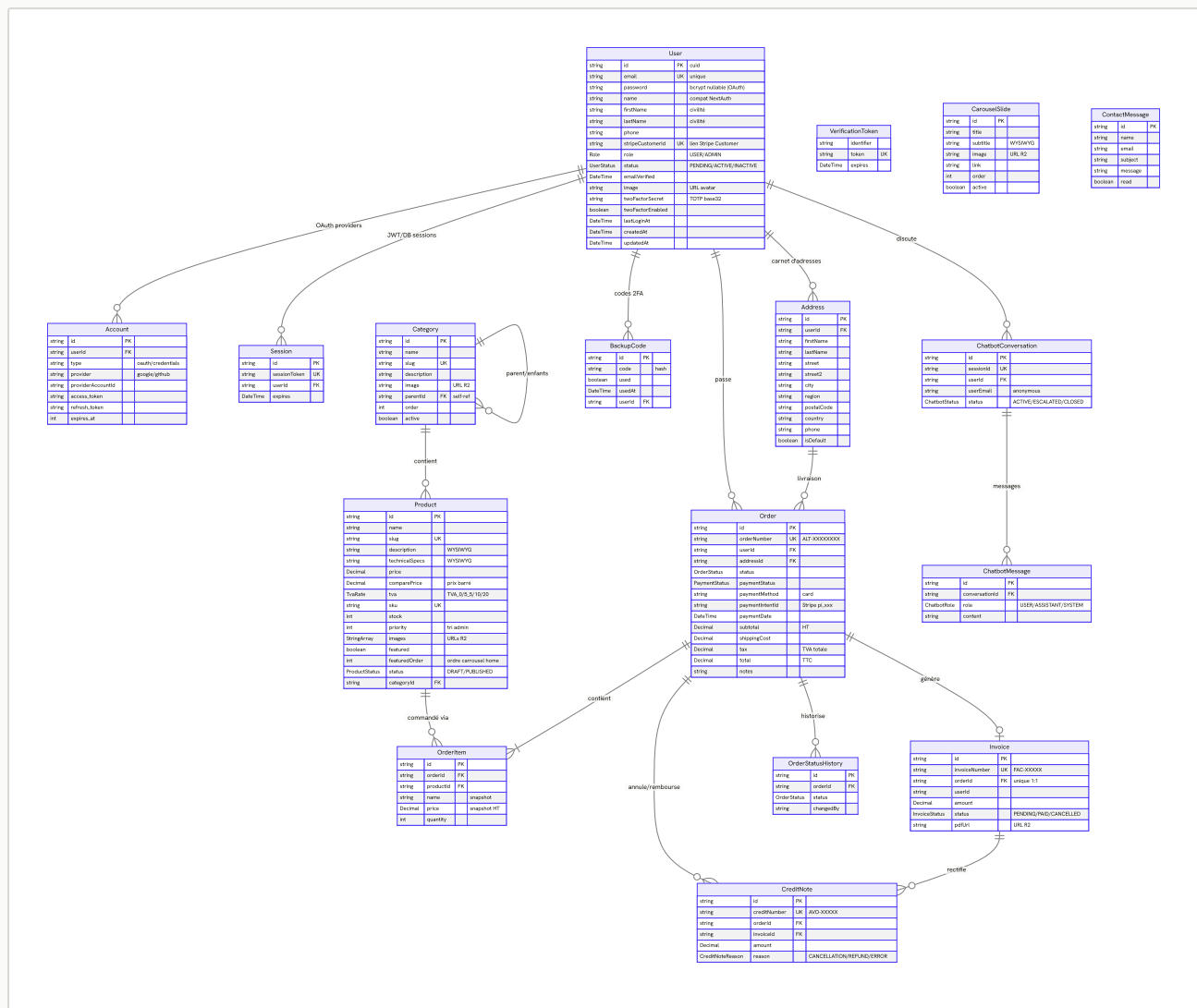
La base de données PostgreSQL contient **15 modèles** et **9 énumérations**, organisés en quatre domaines fonctionnels :

| DOMAINE           | MODÈLES                                                                 |
|-------------------|-------------------------------------------------------------------------|
| Identité et accès | User , Account , Session , VerificationToken , BackupCode               |
| Catalogue         | Category , Product , CarouselSlide                                      |
| Commerce          | Address , Order , OrderItem , OrderStatusHistory , Invoice , CreditNote |
| Communication     | ContactMessage , ChatbotConversation , ChatbotMessage                   |

Toutes les clés primaires sont des **CUID** (Collision-Resistant Unique IDentifier) générés par Prisma — chaînes courtes, URL-safe, triables temporellement.



## § 3.2 Diagramme ERD complet



## § 3.3 Description détaillée des entités

### 3.3.1 User

Cœur du système d'identité. Stocke les utilisateurs B2C, B2B et administrateurs.

| CHAMP            | TYPE    | CONSTRAINTES     | DESCRIPTION                                |
|------------------|---------|------------------|--------------------------------------------|
| <b>id</b>        | String  | PK, CUID auto    | Identifiant unique                         |
| <b>email</b>     | String  | UNIQUE, NOT NULL | Adresse email (login)                      |
| <b>password</b>  | String? | Nullable         | Hash bcrypt (null si OAuth seul)           |
| <b>name</b>      | String? | —                | Champ legacy NextAuth (concaténation auto) |
| <b>firstName</b> | String? | —                | Prénom (formulaire)                        |

| CHAMP                         | TYPE       | CONTRAINTES     | DESCRIPTION                                   |
|-------------------------------|------------|-----------------|-----------------------------------------------|
| <code>lastName</code>         | String?    | —               | Nom (formulaire)                              |
| <code>phone</code>            | String?    | —               | Téléphone (E.164)                             |
| <code>stripeCustomerId</code> | String?    | UNIQUE          | ID Customer Stripe (créé au 1er paiement)     |
| <code>role</code>             | Role       | DEFAULT USER    | USER ou ADMIN                                 |
| <code>status</code>           | UserStatus | DEFAULT PENDING | PENDING (email non vérifié), ACTIVE, INACTIVE |
| <code>emailVerified</code>    | DateTime?  | —               | Date de vérification email                    |
| <code>image</code>            | String?    | —               | URL avatar (OAuth ou upload)                  |
| <code>twoFactorSecret</code>  | String?    | —               | Secret TOTP base32                            |
| <code>twoFactorEnabled</code> | Boolean    | DEFAULT false   | 2FA activée                                   |
| <code>lastLoginAt</code>      | DateTime?  | —               | Dernière connexion (audit)                    |
| <code>createdAt</code>        | DateTime   | DEFAULT now()   | —                                             |
| <code>updatedAt</code>        | DateTime   | @updatedAt      | —                                             |

**Relations :** Account[], Session[], Address[], Order[], BackupCode[], ChatbotConversation[].

#### Règles métier :

- Un utilisateur ADMIN doit obligatoirement avoir activé le 2FA pour accéder à `/admin/*` (forcé par `middleware.ts`).
- La suppression d'un compte cascade sur Account, Session, BackupCode, ChatbotConversation (mise à null pour celle-ci).
- Le champ `stripeCustomerId` n'est créé qu'à la première interaction Stripe (paiement ou enregistrement carte).

### 3.3.2 Account, Session, VerificationToken

Tables imposées par l'Adapter Prisma de NextAuth.

- **Account** stocke les liaisons OAuth ( `provider: 'google' | 'github' + providerAccountId` ).
- **Session** stocke les sessions persistées (utilisé en mode JWT pour la persistance optionnelle).
- **VerificationToken** stocke les tokens de vérification email (TTL 24 h) et de réinitialisation de mot de passe.

### 3.3.3 BackupCode

Codes de récupération 2FA. Générés par lots de 10 lors de l'activation du 2FA. Hashés en base.

Marqués `used: true` après utilisation pour empêcher la réutilisation.

### 3.3.4 Address

Carnet d'adresses utilisateur. Champs conformes au format postal français + extensible international.

| PARTICULARITÉ          | DESCRIPTION                                                             |
|------------------------|-------------------------------------------------------------------------|
| <code>region</code>    | Optionnel — état/province (utile US, CA)                                |
| <code>street2</code>   | Complément (étage, bâtiment)                                            |
| <code>isDefault</code> | Une adresse par défaut maximum par utilisateur (contrainte applicative) |

### 3.3.5 Category

Arborescence de catégories produit. Auto-référencée pour gérer les sous-catégories.

| PARTICULARITÉ         | DESCRIPTION                                                 |
|-----------------------|-------------------------------------------------------------|
| <code>parentId</code> | Null = catégorie racine ; sinon = sous-catégorie            |
| <code>order</code>    | Tri manuel (drag-and-drop dans l'admin)                     |
| <code>active</code>   | Permet de masquer une catégorie sans supprimer ses produits |
| <code>image</code>    | URL R2 (illustration carte catégorie)                       |

Cache Redis : `categories:tree` avec TTL 30 min. Invalidé sur tout CRUD.

### 3.3.6 Product

Référence catalogue.

| CHAMP PARTICULIER                                      | DESCRIPTION                                                                                 |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>description</code> , <code>technicalSpecs</code> | Contenu HTML produit par l'éditeur Tiptap (WYSIWYG), sanitisé via DOMPurify avant insertion |
| <code>price</code> , <code>comparePrice</code>         | <code>Decimal(10, 2)</code> pour précision financière                                       |
| <code>tva</code>                                       | Taux TVA (4 valeurs possibles, voir enum)                                                   |
| <code>sku</code>                                       | Référence interne (Stock Keeping Unit)                                                      |
| <code>stock</code>                                     | Quantité disponible (décrémentée par webhook Stripe)                                        |
| <code>priority</code>                                  | Tri par défaut catalogue (admin)                                                            |
| <code>images</code>                                    | Array d'URLs R2                                                                             |
| <code>featured</code> , <code>featuredOrder</code>     | Carrousel home : produits explicitement mis en avant                                        |
| <code>status</code>                                    | DRAFT (invisible public) ou PUBLISHED                                                       |

Index : `[featured, featuredOrder]` pour requête home rapide.

### 3.3.7 Order

Commande client.

| CHAMP                                                                                     | DESCRIPTION                                                                                 |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>orderNumber</code>                                                                  | Tronqué <code>ALT-</code> + 8 chars depuis Stripe session ID (commit <code>bfd2b84</code> ) |
| <code>paymentIntentId</code>                                                              | Reference Stripe ( <code>pi_XXX</code> ) — utilisé pour les remboursements                  |
| <code>subtotal</code> , <code>shippingCost</code> , <code>tax</code> , <code>total</code> | <code>Decimal(10, 2)</code> — calculés serveur, jamais trustés client                       |
| <code>notes</code>                                                                        | Commentaire client optionnel au checkout                                                    |

Workflow statuts : `PENDING` → `CONFIRMED` → `PROCESSING` → `SHIPPED` → `DELIVERED` ou `CANCELLED`.

### 3.3.8 OrderItem

Lignes de commande. Snapshot du produit au moment de l'achat (`name`, `price`) — protège contre les modifications ultérieures du produit.

### 3.3.9 OrderStatusHistory

Audit trail des changements de statut. Permet de reconstruire la timeline de la commande dans l'admin.

### 3.3.10 Invoice

Facture générée à la confirmation de paiement.

| CHAMP                      | DESCRIPTION                                                                                    |
|----------------------------|------------------------------------------------------------------------------------------------|
| <code>invoiceNumber</code> | Format <code>FAC-AAAAMJJ-XXXX</code> (séquentiel par jour)                                     |
| <code>pdfUrl</code>        | URL R2 du PDF généré par <code>@react-pdf/renderer</code>                                      |
| <code>status</code>        | <code>PENDING</code> → <code>PAID</code> après webhook ou <code>CANCELLED</code> si annulation |

### 3.3.11 CreditNote (Avoir)

Document fiscal d'annulation/remboursement. Référence l'Order ET la Facture. Trois raisons :

`CANCELLATION` (commande annulée avant expédition), `REFUND` (remboursement après livraison), `ERROR` (correction d'erreur de facturation).

### 3.3.12 CarouselSlide

Slides du carrousel d'accueil. Édité dans `/admin/carousel`. Sous-titre en HTML (Tiptap WYSIWYG, sanitisé).

### 3.3.13 ContactMessage

Messages reçus depuis le formulaire de contact. Marquage `read: true` quand consulté dans l'admin. Réponse possible directement depuis la console (envoi via Resend).

### 3.3.14 ChatbotConversation et ChatbotMessage

Persistence des conversations chatbot. Permet à un admin de reprendre une conversation et d'analyser les patterns d'échec du bot.

| STATUT    | SENS                                |
|-----------|-------------------------------------|
| ACTIVE    | Conversation en cours               |
| ESCALATED | Demande explicite de contact humain |
| CLOSED    | Conversation terminée               |

## § 3.4 Énumérations

```
enum Role { USER ADMIN }
enum UserStatus { PENDING ACTIVE INACTIVE }
enum ProductStatus { DRAFT PUBLISHED }
enum TvaRate { TVA_20 TVA_10 TVA_5_5 TVA_0 }
enum OrderStatus { PENDING CONFIRMED PROCESSING SHIPPED DELIVERED CANCELLED }
enum PaymentStatus { PENDING PAID FAILED REFUNDED }
enum InvoiceStatus { PENDING PAID CANCELLED }
enum CreditNoteReason { CANCELLATION REFUND ERROR }
enum ChatbotRole { USER ASSISTANT SYSTEM }
enum ChatbotStatus { ACTIVE ESCALATED CLOSED }
```

Les énumérations PostgreSQL sont utilisées plutôt que des chaînes libres pour garantir l'intégrité référentielle, l'autocomplétion TypeScript et la performance des requêtes filtrées.

### Détail des taux de TVA

| ENUM    | TAUX  | CAS D'USAGE MATÉRIEL MÉDICAL                               |
|---------|-------|------------------------------------------------------------|
| TVA_20  | 20 %  | Taux normal (matériel non médical, instruments génériques) |
| TVA_10  | 10 %  | Taux intermédiaire                                         |
| TVA_5_5 | 5,5 % | Taux réduit (équipements pour personnes handicapées)       |

| ENUM  | TAUX | CAS D'USAGE MATÉRIEL MÉDICAL                         |
|-------|------|------------------------------------------------------|
| TVA_0 | 0 %  | Exonération (dispositifs médicaux remboursables LPP) |

Le calcul de TVA est réalisé via `src/lib/tva-utils.ts` (fonction `calculateTva(price, rate)` ).

### § 3.5 Relations et contraintes

#### Cascade de suppression

| ENTITÉ PARENT       | ENTITÉ ENFANT       | COMPORTEMENT                                   |
|---------------------|---------------------|------------------------------------------------|
| User                | Account             | CASCADE                                        |
| User                | Session             | CASCADE                                        |
| User                | BackupCode          | CASCADE                                        |
| User                | ChatbotConversation | SET NULL (préserve les conversations anonymes) |
| Order               | OrderItem           | CASCADE                                        |
| Order               | OrderStatusHistory  | CASCADE                                        |
| ChatbotConversation | ChatbotMessage      | CASCADE                                        |

User → Address, User → Order, Order → Address, Product → OrderItem, Order → Invoice, Order → CreditNote ne cascaden**t pas** : la suppression d'un User ou Product est bloquée s'il a des commandes (intégrité métier).

#### Contraintes d'unicité

- User.email, User.stripeCustomerId
- Product.slug, Product.sku
- Category.slug
- Order.orderNumber
- Invoice.invoiceNumber, Invoice.orderId (1:1)
- CreditNote.creditNumber
- ChatbotConversation.sessionId
- Account[provider, providerAccountId] (composite)
- VerificationToken[identifiant, token] (composite)

#### Index optimisés

- Product[featured, featuredOrder] — carrousel home

- `BackupCode[userId, used]` — récupération 2FA
- `ChatbotConversation[userId], [status], [createdAt]`
- `ChatbotMessage[conversationId], [createdAt]`

## § 3.6 Migrations Prisma

L'évolution du schéma est versionnée dans `prisma/migrations/`. Chaque migration contient un fichier SQL exécuté en transaction par `prisma migrate deploy`.

### Workflow de migration

```
# Développement (génère + applique + met à jour le client)
npx prisma migrate dev --name <description>

# Production (applique les migrations existantes uniquement)
npx prisma migrate deploy

# Réinitialisation complète (drop + migrate + seed)
npx prisma migrate reset
```

### Seed de données

Le fichier `prisma/seed.ts` peuple la base de tests avec :

- 1 compte admin ( `admin@althea.com` / `Admin123!` — 2FA à configurer manuellement)
- ~10 utilisateurs B2B fictifs
- ~15 catégories (Imagerie, Diagnostic, Premiers secours, etc.)
- ~80 produits avec images
- 3 slides de carrousel

Lancement : `npm run db:seed`.

---





## 4. Fonctionnalités implémentées

---

### § 4.1 Vitrine publique

#### 4.1.1 Page d'accueil ( / )

Composée de cinq sections empilées :

1. **Hero éditorial** ( `src/components/home/hero-editorial.tsx` )
  - Titre principal en `SplitText` (révélation caractère par caractère via Framer Motion)
  - SVG « Blueprint » technique animé (anneaux concentriques, crosshair, graduations)
  - CTA principal en `MagneticButton` (effet d'attraction au survol)
  - Smooth scroll global via Lenis
2. **Carrousel produits mis en avant** ( `src/components/home/cta-products-carousel.tsx` )
  - Affiche uniquement les produits `featured = true` triés par `featuredOrder`
  - Limite : 3 produits maximum (commit `e8d9d24` )
  - Centré si moins de 3 produits (commit `6858005` )
  - Défilement horizontal pin-scroll via GSAP ScrollTrigger sur desktop
  - Drag-to-scroll avec pointer capture sur mobile
  - Parallax inverse sur les images
  - Mobile : snap-scroll natif CSS (pas de GSAP, pour les perfs)
3. **Section ticker** ( `src/components/home/marks-ticker.tsx` )
  - Marquee infini avec marques de confiance (Catalogue indexé, Stock temps réel, Livraison J+2, etc.)
  - Quadruplé en interne pour éviter le gap visible sur grands écrans (commit `c1068bc` )
4. **Carrousel admin** ( `src/components/home/admin-carousel.tsx` )
  - Slides éditoriaux gérés depuis `/admin/carousel`
  - Auto-play via `embla-carousel-autoplay`
  - Inséré entre le hero et le ticker (commit `e2e258b` )
5. **Catégories en grille** ( `src/components/home/categories-grid.tsx` )
  - Top catégories actives, image + lien vers `/products?category=slug`

#### 4.1.2 Catalogue ( `/products` )

- Liste paginée (12 produits par page par défaut)
- Filtres : catégorie, prix min/max, statut (PUBLISHED uniquement en public), tri (récent, prix asc/desc, popularité)
- Grille responsive : 1 colonne mobile, 2 tablette, 3-4 desktop
- Cards éditoriales ( `src/components/products/product-card.tsx` )
  - Format portrait aspect 3/4
  - Image grayscale par défaut, couleur au hover
  - Typographie monospace caps avec tracking
  - Outline plutôt que shadow (style éditorial)

#### 4.1.3 Fiche produit ( `/products/[slug]` )

- Galerie ( `src/components/products/product-gallery.tsx` )
  - Layout 2 colonnes (thumbnails 88px + image principale `1fr` )
  - Si 1 seule image : pleine largeur (commit `eb0c1e8` )
  - Aspect 4/5 portrait par défaut (commit `9ce8951` )
- Informations
  - Nom, prix HT, prix TTC calculé selon taux TVA
  - Description WYSIWYG
  - Spécifications techniques WYSIWYG
  - Stock disponible
  - Bouton « Ajouter au panier » avec sélecteur quantité
- Produits similaires ( `/api/products/[id]/similar` ) — même catégorie, hors produit courant
- JSON-LD `Product` avec `MerchantReturnPolicy` et `OfferShippingDetails` pour le SEO

#### 4.1.4 Recherche ( `/search` )

- Recherche full-text sur `name` , `description` , `sku`
- Auto-complétion (debounce 300 ms) avec dropdown de suggestions
- Facettes : catégorie, prix (range slider), TVA, disponibilité
- Résultats : grille produits + compteur résultats + filtres actifs supprimables

#### 4.1.5 Pages légales

- `/legal/cgu` — Conditions Générales de Vente
- `/legal/mentions-legales` — Mentions légales
- `/legal/privacy` — Politique de confidentialité (RGPD)
- Composant générique `LegalPage` avec sommaire + scrollspy (commit `2166fd3` )

#### 4.1.6 Page contact ( `/contact` )

- Formulaire (nom, email, sujet, message) avec validation Zod
- Consentement RGPD obligatoire (case à cocher)
- Rate limiting : 5 requêtes/heure par IP
- Stockage en `ContactMessage`, notification admin par email

### § 4.2 Espace client

#### 4.2.1 Tableau de bord ( `/account` )

Vue d'ensemble avec :

- Cartes KPI (commandes, total dépensé, dernière commande)
- Dernières commandes (5)
- Adresses récentes
- Style « bento » sombre (commit `7cc0e92` )

#### 4.2.2 Profil ( `/profile` )

- Modification nom, prénom, téléphone, photo
- Changement d'email avec re-vérification
- Changement de mot de passe avec validation CNIL :
  - Minimum 12 caractères
  - 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial
  - (commit `f9f97fe` )

#### 4.2.3 Adresses ( `/addresses` )

- Liste des adresses
- Création/édition avec **autocomplete API BAN** ( `useAddressAutocomplete` )
  - Hook `src/hooks/use-address-autocomplete.ts`
  - Debounce 300 ms, AbortController pour annuler les requêtes obsolètes
  - Auto-fill rue, code postal, ville, pays au clic sur suggestion
- Désignation d'une adresse par défaut

#### 4.2.4 Commandes ( `/orders` )

- Historique paginé groupé par année (commit `f9f97fe` )
- Filtres par statut, période
- Détail commande avec timeline statuts, produits, totaux, lien facture/avoir

#### 4.2.5 Paiement ( `/payments` )

- Liste des moyens de paiement enregistrés (Stripe)
- Ajout via SetupIntent Stripe
- Suppression
- Désignation par défaut

#### 4.2.6 Export RGPD

- Bouton « Exporter mes données » → `/api/profile/export` retourne un JSON complet
- Bouton « Supprimer mon compte » → `/api/profile/delete` (anonymise les commandes, supprime le User)

### § 4.3 Tunnel de commande

#### 4.3.1 Panier ( `/cart` )

- Persistance via cookie côté client (Zustand `cart-store` + middleware persist)
- Items : nom, image, quantité (modifiable), prix, sous-total
- Calcul total dynamique
- Bouton « Procéder au paiement »

#### 4.3.2 Checkout ( `/checkout` )

Multi-étapes :

1. **Adresse de livraison** (existante ou nouvelle avec autocomplete BAN)
2. **Récapitulatif** : sous-total HT + TVA détaillée par taux + frais de port + total TTC
3. **Paiement** : redirection Stripe Checkout

#### 4.3.3 Confirmation ( `/checkout/confirmation` )

- Message de remerciement
- Récapitulatif commande
- Lien téléchargement facture PDF
- (Le PDF est en cours de génération, lien apparaît après webhook traité)

#### 4.3.4 Webhook Stripe

- Endpoint `/api/stripe/webhook` (POST uniquement)
- Vérification de signature obligatoire ( `stripe.webhooks.constructEvent` )
- Événements traités :

- `checkout.session.completed` → confirme l'Order, génère Invoice + PDF, envoie email
- `payment_intent.payment_failed` → marque Order en FAILED
- `charge.refunded` → crée CreditNote

## § 4.4 Back-office d'administration

Accessible sur `/admin/*` uniquement après :

1. Authentification email/mot de passe
2. Rôle ADMIN
3. 2FA TOTP activé
4. Code 2FA validé pour la session

### 4.4.1 Dashboard ( `/admin` )

- KPIs : CA jour/semaine/mois, nombre commandes, clients actifs, panier moyen
- Graphique ventes : courbe sur 7j/30j/90j/1 an (Recharts)
- Répartition catégories : pie chart
- Analyse paniers : taux de conversion, panier moyen, paniers abandonnés
- Alertes stock : produits avec stock < seuil

### 4.4.2 Produits ( `/admin/products` )

- Tableau (TanStack Table) avec tri, filtre, pagination, sélection multiple
- Recherche par nom/SKU
- Filtres : catégorie, statut, prix
- Actions :
  - Créer (formulaire avec uploader R2 multi-images, éditeur Tiptap, calcul TVA)
  - Éditer (mêmes champs)
  - Dupliquer
  - Supprimer (soft : passe en DRAFT)
  - Bulk : changer statut/catégorie en masse
  - Export Excel (commit `0a16fcf` puis `817a2d2` avec ExcelJS)
  - Réorganiser produits mis en avant (drag-and-drop)

### 4.4.3 Catégories ( `/admin/categories` )

- Arborescence avec drag-and-drop (dnd-kit)
- CRUD complet
- Activation/désactivation
- Upload image illustration

#### 4.4.4 Commandes ( /admin/orders )

- Tableau filtré par statut, paiement, date, client
- Détail commande avec timeline complète (OrderStatusHistory)
- Actions :
  - Changer statut (déclenche email client)
  - Générer/régénérer facture
  - Créer avoir (CreditNote)
  - Ajouter note interne
- Export Excel

#### 4.4.5 Utilisateurs ( /admin/users )

- Tableau avec recherche email/nom
- Filtres : rôle, statut, 2FA
- Détail utilisateur avec :
  - Informations personnelles
  - Adresses
  - Historique commandes
  - Activité (dernière connexion)
- Actions : changer rôle, activer/désactiver, réinitialiser 2FA

#### 4.4.6 Carrousel ( /admin/carousel )

- Liste des slides avec drag-and-drop pour réorganiser
- CRUD slide (titre, sous-titre WYSIWYG, image R2, lien CTA, actif)

#### 4.4.7 Messages contact ( /admin/messages )

- Boîte de réception
- Marquage lu/non-lu
- Réponse intégrée (envoi via Resend)
- Export Excel

#### 4.4.8 Avoirs ( /admin/credits )

- CRUD avoirs (CreditNote)
- Génération PDF avoir
- Export Excel

#### 4.4.9 Paramètres ( `/admin/settings` )

- Configuration 2FA admin (initialisation, codes de backup)
- Profil admin

#### 4.4.10 Console chatbot ( `/admin/chatbot` )

- Liste des conversations (filtres ACTIVE/ESCALATED/CLOSED)
- Détail conversation avec historique messages
- Reprise manuelle (changement statut)

### § 4.5 Authentification

#### 4.5.1 Inscription ( `/register` )

- Formulaire : prénom, nom, email, mot de passe, confirmation, consentement RGPD
- Validation Zod (schéma `src/lib/validators/auth.ts`)
- Création User en `status: PENDING`
- Envoi email de vérification (token TTL 24 h en Redis)
- Redirection vers page « Vérifiez votre email »

#### 4.5.2 Vérification email ( `/verify-email?token=xxx` )

- Récupère token depuis URL
- Vérifie validité Redis
- Active le compte ( `status: ACTIVE` , `emailVerified: now()` )
- Redirection vers login (utilise `NEXTAUTH_URL` au lieu de `request.url` — commit `234d7c8` )

#### 4.5.3 Connexion ( `/login` )

- Email + mot de passe (Credentials Provider NextAuth)
- OAuth Google (bouton)
- OAuth GitHub (bouton)
- Lien « Mot de passe oublié »
- `allowDangerousEmailAccountLinking: true` pour permettre la liaison multi-providers sur le même email

#### 4.5.4 Mot de passe oublié ( `/forgot-password` )

- Saisie email
- Toujours retourne 200 (anti-énumération)
- Si email existe, génère token (TTL 1 h), envoie email avec lien `/reset-password?token=xxx`

### 4.5.5 Réinitialisation ( `/reset-password` )

- Vérifie token Redis
- Saisie nouveau mot de passe (validation CNIL)
- Met à jour User.password (bcrypt)
- Invalide le token

### 4.5.6 Configuration 2FA ( `/admin/settings/2fa` )

Process en 3 étapes :

1. **Setup** ( `POST /api/auth/2fa/setup` )
  - Génère secret base32 ( `authenticator.generateSecret()` )
  - Stocke temporairement en Redis (clé `2fa-setup:{userId}`, TTL 10 min)
  - Génère URL otpauth + QR code (`qrcode.react`)
2. **Vérification** ( `POST /api/auth/2fa/verify` )
  - Récupère secret depuis Redis
  - Vérifie code TOTP saisi
  - Si valide : sauvegarde secret en base, `twoFactorEnabled: true`
  - Génère 10 backup codes (hashés)
  - Affiche les codes une seule fois
3. **Verify session** (à chaque login admin)
  - Middleware redirige vers `/admin/verify-2fa`
  - Saisie code (6 chiffres) ou backup code
  - Met à jour la session JWT ( `twoFactorVerified: true` )

## § 4.6 Internationalisation

### 4.6.1 Architecture next-intl

- Routing basé cookie (et non `[locale]/` dans l'URL) pour ne pas casser NextAuth
- Trois locales supportées : `fr` (défaut), `en`, `ar`
- Traductions dans `src/i18n/locales/{fr,en,ar}.json`

### 4.6.2 Composant LanguageSwitcher

- Compact dans le header (commit `bd6a6da` : déplacé dans menu mobile pour aérer)
- Cookie persistance immédiate sans reload



### 4.6.3 Support RTL (arabe)

- Fonction `getDirection(locale)` retourne `'rtl'` pour `ar`, `'ltr'` sinon
- Appliquée sur `<html dir={dir}>`
- Police Noto Sans Arabic chargée pour `ar`

## § 4.7 Chatbot IA

### 4.7.1 Backend ( `/api/chat` )

- Streaming SSE (Server-Sent Events) depuis OpenAI
- Modèle `gpt-4o-mini`
- Validation Zod des messages entrants
- Rate limiting 5 req/min par IP via Redis

### 4.7.2 Frontend ( `src/components/chatbot/` )

- Bubble flottante en bas à droite
- Historique persistant en `localStorage`
- Streaming token-par-token affiché en temps réel
- Style cohérent avec le design system éditorial

### 4.7.3 Persistance et console admin

- Conversations sauveées en `ChatbotConversation` + `ChatbotMessage` (commit `9f58b98` )
- Console admin ( `/admin/chatbot` ) pour visualisation et reprise

## § 4.8 Emails transactionnels

Tous les emails sont envoyés via **Resend** (templates React).

| EMAIL                      | DÉCLENCHEUR                                            | TEMPLATE                                       |
|----------------------------|--------------------------------------------------------|------------------------------------------------|
| Bienvenue + vérification   | POST <code>/api/auth/register</code>                   | <code>welcome-verify.tsx</code>                |
| Mot de passe oublié        | POST <code>/api/auth/forgot-password</code>            | <code>reset-password.tsx</code>                |
| Confirmation commande      | Webhook Stripe <code>checkout.session.completed</code> | <code>order-confirmation.tsx</code>            |
| Facture                    | Génération facture                                     | <code>invoice.tsx</code> (PDF en pièce jointe) |
| Avoir                      | Création CreditNote                                    | <code>credit-note.tsx</code>                   |
| Changement statut commande | Action admin                                           | <code>order-status-update.tsx</code>           |

| EMAIL           | DÉCLENCHEUR  | TEMPLATE                       |
|-----------------|--------------|--------------------------------|
| Réponse contact | Action admin | <code>contact-reply.tsx</code> |

Templates refondus à la charte Althea (commit `bb7fd07`, `e7cf4f5`).

## § 4.9 SEO

### 4.9.1 Métadonnées

- `metadata` Next.js 16 sur chaque page
- Open Graph + Twitter Cards
- `hreflang` alternates pour `fr-FR`, `en-US`, `ar`, `x-default`

### 4.9.2 JSON-LD enrichi

- Pages globales : `Organization` + `LocalBusiness` + `Store` (Knowledge Panel Google)
- Pages produits : `Product` + `MerchantReturnPolicy` + `OfferShippingDetails`
- Pages catégories : `CollectionPage` + `BreadcrumbList`

### 4.9.3 Sitemap dynamique

- `src/app/sitemap.ts` génère `sitemap.xml` depuis Prisma
- Bug critique corrigé : utilisait `product.id` au lieu de `product.slug` (commit `7c92897`)

### 4.9.4 OG Image dynamique

- Screenshot Playwright du hero + blueprint stocké sur R2 CDN
- Mise à jour à chaque redéploiement (commit `e0d63fd`)

### 4.9.5 PWA

- `manifest.webmanifest` configuré
- Favicon Althea aux couleurs de la charte (commit `2d9f2c9`)

## § 4.10 Cas d'usage détaillés

### Cas 1 — Nouvel utilisateur effectue sa première commande

| Utilisateur                         | Système |
|-------------------------------------|---------|
| Visite <code>althea.vjuya.me</code> |         |

|                         |                                          |
|-------------------------|------------------------------------------|
| ----->>                 | Affiche home (RSC, cache 60s)            |
| Recherche "stéthoscope" |                                          |
| ----->>                 | GET /api/search → cache Redis ou Prisma  |
| Clic sur produit        |                                          |
| ----->>                 | Affiche /products/[slug] (RSC + JSON-LD) |
| Ajoute au panier        |                                          |
| (qty=2)                 | cart-store.add() – persisté cookie       |
| Clic "Procéder au       |                                          |
| paiement"               |                                          |
| ----->>                 | Redirige /checkout                       |
|                         | middleware: auth requis → /login         |
| S'inscrit (register)    |                                          |
| ----->>                 | POST /api/auth/register                  |
|                         | Crée User PENDING, envoie email          |
| Clique lien email       |                                          |
| ----->>                 | GET /verify-email?token=xxx              |
|                         | Active User, redirige login              |
| Se connecte             |                                          |
| ----->>                 | POST /api/auth/callback/credentials      |
|                         | JWT cookie httpOnly set                  |
| Redirige checkout       |                                          |
| Saisit adresse          |                                          |
| (autocomplete BAN)      |                                          |
| Confirme                |                                          |
| ----->>                 | POST /api/stripe/checkout                |
|                         | Crée Order PENDING + Items + History     |
|                         | Décrémente stock                         |
|                         | Crée Stripe Checkout Session             |
|                         | Renvoie URL                              |
| Redirige Stripe         |                                          |
| Paie CB                 |                                          |
|                         | Stripe envoie webhook                    |
|                         | Vérifie signature                        |
|                         | Order → CONFIRMED, PAID                  |
|                         | Crée Invoice                             |
|                         | Génère PDF facture                       |
|                         | Upload R2                                |
|                         | Envoie email confirmation                |
| Redirige /confirmation  |                                          |
| (Stripe success URL)    |                                          |
| Reçoit email avec PDF   |                                          |

## Cas 2 — Admin met à jour un produit en rupture

|             |                                   |
|-------------|-----------------------------------|
| Admin       | Système                           |
| Login + 2FA |                                   |
| ----->>     | Vérifie tout, accorde accès admin |

|  |                           |                               |
|--|---------------------------|-------------------------------|
|  |                           |                               |
|  | Reçoit alerte stock bas   |                               |
|  | (dashboard /admin)        |                               |
|  | Clique produit            |                               |
|  | ----->>                   | GET /api/admin/products/[id]  |
|  | Met à jour stock          |                               |
|  | + nouveau prix            |                               |
|  | ----->>                   | PUT /api/admin/products/[id]  |
|  |                           | Validation Zod (PATCH partiel |
|  |                           | accepté – commit 46b641f)     |
|  |                           | Met à jour Product            |
|  |                           | Invalide cache Redis          |
|  |                           |                               |
|  | Reçoit confirmation toast |                               |

Cas 3 — Client demande un remboursement

|        |                          |                              |
|--------|--------------------------|------------------------------|
| Client | Admin                    | Système                      |
|        |                          |                              |
|        | Email service client     |                              |
|        | (commande #ALT-XYZ)      |                              |
|        |                          |                              |
|        | Reçoit message           |                              |
|        | Va sur /admin/orders/XYZ |                              |
|        |                          |                              |
|        | Vérifie commande         |                              |
|        | Clique "Rembourser"      |                              |
|        | ----->>                  | Stripe refund API            |
|        |                          | charge.refunded webhook      |
|        |                          | Crée CreditNote              |
|        |                          | (reason=REFUND)              |
|        |                          | Order paymentStatus=REFUNDED |
|        |                          | Génère PDF avoir             |
|        |                          | Envoie email avoir           |
|        |                          |                              |
|        | Reçoit email avoir PDF   |                              |



## 5. Structure du code

### § 5.1 Arborescence du projet

```

althea-systems/
├── .github/
│   └── workflows/
│       └── ci.yml                # Pipeline CI (lint + typecheck + build)
├── .nvmrc                       # Node 20
├── docker/
│   ├── Dockerfile               # Multi-stage (deps → builder → runner)
│   ├── docker-compose.yml       # Stack locale (app + postgres + redis)
│   └── query/                   # Scripts SQL d'init
├── docs/                        # Documentation (ce dossier)
├── e2e/                         # Tests Playwright
│   ├── home.spec.ts
│   ├── product-flow.spec.ts
│   ├── cart.spec.ts
│   ├── checkout.spec.ts
│   └── search.spec.ts
├── __tests__/                   # Tests Vitest
│   ├── unit/                   # Tests unitaires
│   └── integration/            # Tests d'intégration API
├── prisma/
│   ├── schema.prisma           # 15 modèles, 9 enums
│   ├── migrations/             # Migrations versionnées
│   └── seed.ts                 # Seed B2B médical
├── public/                     # Assets statiques (favicon, fonts, images publiques)
├── scripts/                    # Scripts utilitaires
│   └── security-audit.ts        # Audit sécurité custom (génère security-report.md)
├── src/
│   ├── middleware.ts            # Middleware Edge – auth + headers + skip
│   └── app/
│       ├── layout.tsx           # Layout racine (html, body, fonts, providers)
│       ├── globals.css          # Tailwind + @theme + tokens
│       ├── sitemap.ts           # sitemap.xml dynamique
│       ├── robots.ts            # robots.txt
│       ├── (site)/              # Pages publiques (route group)
│       │   ├── layout.tsx       # Layout site (Header + Footer)
│       │   ├── page.tsx         # Home
│       │   ├── products/
│       │   ├── categories/
│       │   ├── search/
│       │   ├── cart/
│       │   ├── checkout/
│       │   ├── contact/
│       │   ├── about/
│       │   └── legal/
│       ├── (auth)/              # Login, register, reset, verify
│       ├── (account)/           # Profil, commandes, adresses, paiements
│       ├── admin/               # Back-office (protégé middleware)
│       │   ├── layout.tsx       # Shell admin avec sidebar
│       │   ├── page.tsx         # Dashboard
│       │   ├── products/
│       │   ├── categories/
│       │   └── orders/

```

```

├─ users/
├─ carousel/
├─ messages/
├─ credits/
├─ chatbot/
├─ settings/
├─ verify-2fa/
├─ api/ # 72 routes API
├─   ├─ auth/
├─   ├─ products/
├─   ├─ categories/
├─   ├─ orders/
├─   ├─ stripe/
├─   ├─ users/
├─   ├─ addresses/
├─   ├─ profile/
├─   ├─ cart/
├─   ├─ carousel/
├─   ├─ contact/
├─   ├─ chat/
├─   ├─ invoices/
├─   ├─ credits/
├─   ├─ upload/
├─   ├─ images/
├─   ├─ search/
├─   ├─ stats/
├─   ├─ admin/ # Routes admin (KPIs, exports, bulk)
├─   └─ docs/ # Swagger UI
├─ docs/ # Page /docs (Swagger)
├─ components/ # 131 composants
├─   ├─ ui/ # shadcn primitives (Button, Dialog, etc.)
├─   ├─ layout/ # Header, Footer, MobileMenu, Navigation
├─   ├─ home/ # Hero, Carrousels, Ticker, Categories
├─   ├─ products/ # Card, Gallery, FilterBar
├─   ├─ cart/ # CartItem, CartSummary
├─   ├─ checkout/ # CheckoutForm, AddressForm, OrderSummary
├─   ├─ account/ # Bento dashboard, ProfileForm, etc.
├─   ├─ auth/ # LoginForm, RegisterForm, etc.
├─   ├─ admin/ # Shell, DataTable, formulaires CRUD
├─   ├─ search/ # SearchBar, SearchResults, Filters
├─   ├─ contact/ # ContactForm
├─   ├─ chatbot/ # ChatBubble, ChatWindow, MessageList
├─   ├─ seo/ # JsonLd, Breadcrumb
├─   ├─ shared/ # Breadcrumb, Pagination, EmptyState
├─   ├─ tva/ # PriceDisplay (HT, TTC, TVA breakdown)
├─   ├─ legal/ # LegalPage avec scrollspy
├─   └─ providers/ # SessionProvider, ThemeProvider, etc.
├─ hooks/ # Custom hooks
├─   ├─ use-auth.ts
├─   ├─ use-cart.ts
├─   ├─ use-debounce.ts
├─   ├─ use-search.ts
├─   └─ use-address-autocomplete.ts
├─ i18n/
├─   ├─ config.ts # next-intl config
├─   └─ locales/ # fr.json, en.json, ar.json
├─ lib/ # Couche services
├─   ├─ auth.ts # NextAuth config (providers, callbacks)
├─   ├─ prisma.ts # Client Prisma singleton
├─   ├─ redis.ts # Client Redis + helpers cache
├─   ├─ rate-limit.ts # Rate limiting helpers
├─   ├─ stripe.ts # SDK Stripe (lazy loading)
├─   ├─ email.ts # Templates Resend
├─   ├─ r2.ts # S3 SDK pour Cloudflare R2
├─   └─ pdf.tsx # Génération PDF (factures, avoirs)

```

```

├── tva-utils.ts      # Calculs TVA français
├── currency.ts       # Formatage devises
├── csv-export.ts     # Export CSV admin
├── export.ts         # Export Excel (ExcelJS)
├── analytics.ts      # Helpers analytics dashboard
├── backup-codes.ts   # Génération/vérification codes 2FA
├── cart-cookie.ts    # Persistance panier cookie
├── chatbot-storage.ts # Persistance conversations
├── image-optimization.ts
├── notifications.ts
├── security.ts       # Helpers CSP, sanitization
├── env.ts            # Validation variables env
├── utils.ts          # Helpers généraux (cn, formatDate, etc.)
├── admin/
│   └── mock-data.ts  # Mocks pour dev (à retirer)
├── logger/
│   ├── index.ts      # Winston instance
│   ├── sections.ts   # Loggers par section (auth, api, etc.)
│   ├── messages.ts   # Messages standardisés
│   ├── api-wrapper.ts # withApiLogger HOF
│   └── exports.ts     # Re-exports
├── utils/
│   └── tva.ts         # Utilitaires TVA spécifiques
├── validators/       # Schémas Zod
│   ├── auth.ts
│   ├── product.ts
│   ├── order.ts
│   ├── contact.ts
│   └── common.ts
├── stores/           # Zustand stores
│   ├── auth-store.ts
│   ├── cart-store.ts
│   └── ui-store.ts
├── styles/
├── tests/            # Tests inline (peu utilisés)
├── types/            # Types TypeScript globaux
├── .env.example      # Template variables env
├── .gitignore
├── .dockerignore
├── CHANGELOG.md
├── CONTRIBUTING.md
├── Dockerfile        # Dockerfile racine (Dokploy auto-detect)
├── README.md         # 31 Ko, doc d'entrée projet
├── components.json   # Config shadcn
├── eslint.config.mjs  # Config ESLint plat (v9)
├── next.config.ts    # Config Next.js (images, headers)
├── package.json
├── package-lock.json
├── playwright.config.ts # Config E2E
├── postcss.config.mjs # Tailwind 4 plugin
├── tsconfig.json     # TypeScript strict
├── vitest.config.ts  # Config tests
└── vitest.setup.ts   # Setup global tests

```

## § 5.2 Organisation des dossiers

L'organisation suit deux principes :



### 5.2.1 Route groups Next.js

`src/app/(site)/`, `(auth)/`, `(account)/`, `admin/` sont des **route groups** : les parenthèses indiquent à Next.js de ne pas inclure le segment dans l'URL, tout en permettant un layout dédié par groupe.

- `(site)` partage Header + Footer publics
- `(auth)` partage le layout split avec hero panel
- `(account)` partage la sidebar compte client
- `admin` partage le shell admin avec sidebar et header dédiés

### 5.2.2 Composants colocalisés par domaine

Plutôt qu'un gros dossier `components/` à plat, on regroupe par domaine fonctionnel :

`components/cart/`, `components/admin/`, `components/products/`. Cela facilite la lecture (« où est le formulaire d'adresse ? » → `components/checkout/address-form.tsx`).

Les **composants UI génériques** (Button, Dialog, Input, Label) restent dans `components/ui/` (généré par shadcn).

## § 5.3 Conventions de nommage

| ÉLÉMENT                  | CONVENTION                                  | EXEMPLE                                        |
|--------------------------|---------------------------------------------|------------------------------------------------|
| Fichiers React           | kebab-case                                  | <code>product-card.tsx</code>                  |
| Composants               | PascalCase                                  | <code>export function ProductCard()</code>     |
| Hooks                    | camelCase, préfixe <code>use</code>         | <code>useAddressAutocomplete()</code>          |
| Services lib             | kebab-case                                  | <code>tva-utils.ts</code>                      |
| Stores Zustand           | kebab-case <code>-store</code>              | <code>cart-store.ts</code>                     |
| Routes API               | kebab-case                                  | <code>/api/admin/products/bulk/route.ts</code> |
| Composants serveur (RSC) | Sans <code>'use client'</code>              | Par défaut                                     |
| Composants client        | <code>'use client'</code> en première ligne | Formulaires interactifs, hooks d'état          |
| Types                    | PascalCase, interface ou type               | <code>interface ProductWithCategory</code>     |
| Constantes               | SCREAMING_SNAKE_CASE                        | <code>MAX_FILE_SIZE</code>                     |
| Fonctions                | camelCase                                   | <code>calculateTva(price, rate)</code>         |

## § 5.4 Couche de services ( `src/lib/` )

Chaque service encapsule une responsabilité unique.

### 5.4.1 `auth.ts` — Configuration NextAuth

Centralise tous les providers, callbacks, options de session. Exporte `authOptions` utilisé partout via `getSession(authOptions)`.

### 5.4.2 `prisma.ts` — Singleton Prisma

```
import { PrismaClient } from '@prisma/client';
const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Pattern singleton qui évite la création de multiples connexions en hot-reload développement.

### 5.4.3 `redis.ts` — Client Redis avec lazy init

Initialisation différée (créée à la première utilisation) pour éviter les crashes au build (ex : CI sans Redis disponible). L'instance Proxy initialement utilisée a été remplacée par un objet wrapper explicite après un bug en production (commit `8381878`, voir section 6.9).

### 5.4.4 `stripe.ts` — Stripe lazy loading

```
let stripeInstance: Stripe | null = null;
export function getStripe(): Stripe {
  if (!stripeInstance) {
    if (!process.env.STRIPE_SECRET_KEY) {
      throw new Error('STRIPE_SECRET_KEY is not defined');
    }
    stripeInstance = new Stripe(process.env.STRIPE_SECRET_KEY, {
      apiVersion: '2025-11-17.clover',
    });
  }
  return stripeInstance;
}
```

Pattern singleton évitant l'init au build time.

### 5.4.5 `email.ts` — Templates Resend

Wrapper qui prend un type d'email + données, charge le template React correspondant et l'envoie via Resend.

### 5.4.6 `r2.ts` — Upload S3-compatible

Fonctions :

- `uploadImage(file, folder)` — sanitize nom de fichier (regex `/[^\\w.-]/g`), génère clé unique, upload, retourne URL publique
- `deleteImage(url)` — extrait clé de l'URL, supprime sur R2
- Validation MIME (JPEG, PNG, WebP, GIF) et taille (5 Mo max)

### 5.4.7 `pdf.tsx` — Génération React-PDF

Composants React rendus en PDF via `@react-pdf/renderer` :

- `<InvoicePdf order={order} />` → facture
- `<CreditNotePdf creditNote={...} />` → avoir

Utilise les couleurs Althea (electric-indigo, shadow-grey), la typo IBM Plex Mono pour les références (commit `b2cae49`).

### 5.4.8 `validators/` — Schémas Zod

Schémas partagés frontend (React Hook Form via `@hookform/resolvers/zod`) et backend (validation des bodies API).

## § 5.5 Gestion des erreurs

### 5.5.1 Approche générale

Trois niveaux d'erreurs :

1. **Erreurs de validation** → 400 Bad Request avec détails Zod
2. **Erreurs d'autorisation** → 401 Unauthorized ou 403 Forbidden
3. **Erreurs serveur** → 500 Internal Server Error (loggué, message générique au client)

### 5.5.2 Pattern dans les routes API

```
import { withApiLogger } from '@lib/logger/api-wrapper';
import { z } from 'zod';

const bodySchema = z.object({
  name: z.string().min(2),
  price: z.number().positive(),
});

export const POST = withApiLogger(async (req: Request) => {
  // 1. Auth
  const session = await getServerSession(authOptions);
  if (!session || session.user.role !== 'ADMIN') {
```

```

    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  // 2. Validation
  const body = await req.json();
  const parsed = bodySchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json(
      { error: 'Validation failed', issues: parsed.error.issues },
      { status: 400 }
    );
  }

  // 3. Logique métier (try/catch)
  try {
    const product = await prisma.product.create({ data: parsed.data });
    await invalidateCache('products:*');
    return NextResponse.json(product, { status: 201 });
  } catch (e) {
    apilogger.error('Failed to create product', { error: e });
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
});

```

### 5.5.3 Erreurs côté client

- React Error Boundaries ( `error.tsx` Next.js) sur chaque route group
- Page `not-found.tsx` custom (404 brutalist commit `7cc0e92`)
- Toasts Sonner pour feedback utilisateur
- Sentry envisagé (roadmap)

## § 5.6 Validation des entrées

Toutes les entrées utilisateur passent par **Zod** avant traitement.

**Exemple : validateur produit** ( `src/lib/validators/product.ts` )

```

import { z } from 'zod';

export const productCreateSchema = z.object({
  name: z.string().min(2).max(200),
  slug: z.string().regex(/^[a-z0-9-]+$/),
  description: z.string().max(10000).optional(),
  price: z.number().positive().multipleOf(0.01),
  comparePrice: z.number().positive().optional(),
  tva: z.enum(['TVA_20', 'TVA_10', 'TVA_5_5', 'TVA_0']),
  sku: z.string().optional(),
  stock: z.number().int().min(0),
  images: z.array(z.string().url()).max(10),
  featured: z.boolean().default(false),
  categoryId: z.string().cuid(),
  status: z.enum(['DRAFT', 'PUBLISHED']).default('DRAFT'),
});

```

```
export type ProductCreate = z.infer<typeof productCreateSchema>;
```

## Sanitization HTML (DOMPurify)

Tous les contenus WYSIWYG (descriptions produit, sous-titre carrousel) sont sanitisés via DOMPurify côté serveur **avant** insertion en base, pour éliminer toute injection XSS (commit `817a2d2`).

## § 5.7 Logging structuré (`src/lib/logger/`)

### 5.7.1 Architecture Winston

- **Console transport** (toujours actif) — sortie colorée
- **File transports** désactivés en production conteneurisée (commit `8e7ec7f`) car le user `nextjs` (UID 1001) n'a pas les droits d'écriture sur `/app`
- En dev : `logs/combined.log` + `logs/error.log` avec rotation 5 Mo, 5 fichiers max

### 5.7.2 Loggers par section

```
import { authLogger, apiLogger, productLogger, uploadLogger } from '@lib/logger';

apiLogger.info('Product created', { productId: product.id, userId: session.user.id });
authLogger.warn('Failed login attempt', { email });
```

Chaque logger ajoute un champ `section` dans les logs structurés JSON.

### 5.7.3 Wrapper API

```
import { withApiLogger } from '@lib/logger/api-wrapper';

export const GET = withApiLogger(async (req) => {
  // Logue automatiquement : méthode, URL, durée, status
  return NextResponse.json({ ok: true });
});
```

## § 5.8 Tests

### 5.8.1 Tests unitaires (Vitest)

- Configuration : `vitest.config.ts` + `vitest.setup.ts`
- Cible : helpers (`tva-utils`, `currency`, `validators`), composants isolés
- Lancement : `npm test` (mode watch), `npm run test:coverage`

### 5.8.2 Tests d'intégration (Vitest)

- Cible : endpoints API critiques (auth, checkout)
- Base de test PostgreSQL (commit `103e2bc`)
- Lancement : `npm run test:integration`

### 5.8.3 Tests E2E (Playwright)

- Cinq spécifications dans `e2e/`
    - `home.spec.ts` — chargement, sections, accessibilité
    - `product-flow.spec.ts` — recherche → fiche produit → ajout panier
    - `cart.spec.ts` — modification quantités, suppression
    - `checkout.spec.ts` — saisie adresse, redirection Stripe
    - `search.spec.ts` — recherche, filtres
  - Configuration : `playwright.config.ts` (Chromium, Firefox, WebKit)
  - Lancement : `npm run test:e2e`
-



## 6. Sécurité

### § 6.1 Modèle de menaces

Application e-commerce hébergeant des données personnelles (RGPD), des moyens de paiement (PCI-DSS) et des données métier sensibles (CA, clients). Les menaces principales identifiées :

| MENACE                  | SURFACE D'ATTAQUE                      | MITIGATION                                       |
|-------------------------|----------------------------------------|--------------------------------------------------|
| Vol de credentials      | Login, formulaires                     | Bcrypt, rate limiting, 2FA admin                 |
| Vol de session          | Cookies                                | httpOnly, Secure, SameSite                       |
| XSS                     | Contenus WYSIWYG, params URL           | DOMPurify, CSP, escape par défaut React          |
| CSRF                    | Formulaires                            | NextAuth CSRF token, SameSite                    |
| SQL injection           | Toutes routes                          | Prisma (queries paramétrées)                     |
| Injection NoSQL         | —                                      | Pas de NoSQL utilisé                             |
| Brute-force login       | <code>/api/auth/*</code>               | Rate limit 5/min IP                              |
| Énumération emails      | <code>/api/auth/forgot-password</code> | Réponse 200 systématique                         |
| Faux webhooks Stripe    | <code>/api/stripe/webhook</code>       | Vérification signature HMAC                      |
| Élévation privilèges    | Routes admin                           | Middleware role + 2FA                            |
| Données exposées        | API responses                          | Whitelist champs, pas de mot de passe en réponse |
| MITM                    | HTTP                                   | HTTPS forcé, HSTS                                |
| Dépendances vulnérables | npm                                    | npm audit hebdomadaire, CodeQL                   |

### § 6.2 Authentification

#### 6.2.1 Stratégie

Sessions **JWT stateless** stockées en cookie `httpOnly`, `Secure`, `SameSite=Lax`. Durée de vie : **30 jours** (rolling).

Le JWT contient : `id`, `email`, `role`, `firstName`, `lastName`, `image`, `twoFactorEnabled`, `twoFactorVerified`.



### 6.2.2 Mots de passe

- Hash `bcrypt`, salt rounds = 10
- Validation CNIL (commit `f9f97fe`) :
  - Minimum 12 caractères
  - 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial

### 6.2.3 OAuth providers

- Google OAuth 2.0
- GitHub OAuth
- `allowDangerousEmailAccountLinking: true` — permet à un même utilisateur de lier email/password + Google + GitHub sur le même compte (validation du même email)

### 6.2.4 Vérification email

Token unique stocké en Redis (TTL 24 h), envoyé par email à l'inscription. Compte non utilisable tant que `emailVerified` est null.

### 6.2.5 2FA TOTP

- Obligatoire pour les administrateurs
- Optionnel pour les clients
- Implémentation via `otplib` (TOTP RFC 6238)
- Setup en 3 étapes (voir §4.5.6)
- 10 backup codes générés à l'activation, hashés en base, marqués utilisés après usage
- Window de tolérance : 1 (accepte le code en cours, le précédent et le suivant —  $\pm 30s$ )

### 6.2.6 Session lifecycle

|             |                                                                       |
|-------------|-----------------------------------------------------------------------|
| Login       | → JWT créé ( <code>twoFactorVerified=false</code> )                   |
| Page admin  | → Middleware redirige <code>/admin/verify-2fa</code>                  |
| Saisie code | → <code>session.update()</code> — <code>twoFactorVerified=true</code> |
| Logout      | → Cookie supprimé                                                     |
| Inactivité  | → JWT expire après 30 jours                                           |

## § 6.3 Autorisation

### 6.3.1 Middleware (`src/middleware.ts`)

Exécuté sur l'Edge Runtime avant chaque requête (sauf assets et API).

```

export default async function middleware(req: NextRequest) {
  const token = await getToken({ req, secret: process.env.NEXTAUTH_SECRET });
  const { pathname } = req.nextUrl;

  // Admin routes
  if (pathname.startsWith('/admin')) {
    if (!token) return NextResponse.redirect(new URL('/login?callbackUrl=' + pathname, req.url));
    if (token.role !== 'ADMIN') return NextResponse.redirect(new URL('/', req.url));
    if (!token.twoFactorEnabled) return NextResponse.redirect(new URL('/admin/settings/2fa', req.url));
    if (!token.twoFactorVerified && pathname !== '/admin/verify-2fa') {
      return NextResponse.redirect(new URL('/admin/verify-2fa', req.url));
    }
  }

  // Account routes
  if (['/profile', '/orders', '/addresses', '/payments'].some(p => pathname.startsWith(p))) {
    if (!token) return NextResponse.redirect(new URL('/login?callbackUrl=' + pathname, req.url));
  }

  // Auth routes – redirect if already logged in
  if (['/login', '/register'].includes(pathname) && token) {
    return NextResponse.redirect(new URL('/', req.url));
  }

  // Add security headers
  const res = NextResponse.next();
  res.headers.set('X-Frame-Options', 'DENY');
  res.headers.set('X-Content-Type-Options', 'nosniff');
  res.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin');
  return res;
}

```

### 6.3.2 Vérification dans les routes API

Chaque route mutante vérifie :

```

const session = await getServerSession(authOptions);
if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
if (route.requiresAdmin && session.user.role !== 'ADMIN') {
  return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
}

```

### 6.3.3 Vérifications métier

- Une commande ne peut être consultée que par son propriétaire ou un admin (vérifié dans `/api/orders/[id]`)
- Une adresse ne peut être modifiée que par son propriétaire (vérifié dans `/api/addresses/[id]`)
- Un utilisateur ne peut pas se promouvoir admin (route `/api/users/[id]` filtre les champs modifiables)

## § 6.4 Validation et sanitization

- Toutes les entrées validées par Zod (voir §5.6)
- Contenus HTML WYSIWYG sanitizés par DOMPurify côté serveur (descriptions produit, sous-titres carrousel)
- Noms de fichiers uploadés sanitizés par regex `/[^\\w.-]/g` (commit gestion R2)
- Output React échappé par défaut (pas de `dangerouslySetInnerHTML` sur du contenu utilisateur sans sanitize)

## § 6.5 Headers de sécurité

Configurés dans `src/middleware.ts` et `src/lib/security.ts`.

| HEADER                                 | VALEUR                                                | EFFET                                         |
|----------------------------------------|-------------------------------------------------------|-----------------------------------------------|
| <code>X-Frame-Options</code>           | <code>DENY</code>                                     | Empêche l'iframe de l'app (anti-clickjacking) |
| <code>X-Content-Type-Options</code>    | <code>nosniff</code>                                  | Empêche le MIME sniffing                      |
| <code>Referrer-Policy</code>           | <code>strict-origin-when-cross-origin</code>          | Limite la fuite d'URL référente               |
| <code>Strict-Transport-Security</code> | <code>max-age=31536000; includeSubDomains</code>      | Force HTTPS pendant 1 an                      |
| <code>X-XSS-Protection</code>          | <code>1; mode=block</code>                            | Filtre XSS legacy navigateur                  |
| <code>Permissions-Policy</code>        | <code>camera=(), microphone=(), geolocation=()</code> | Refuse capabilities non utilisées             |
| <code>Content-Security-Policy</code>   | Voir ci-dessous                                       | Restreint sources de chargement               |

### CSP en production

```
default-src 'self';
script-src 'self' 'unsafe-inline' 'unsafe-eval' https://js.stripe.com
https://www.googletagmanager.com;
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
img-src 'self' data: blob: https://pub-*.r2.dev https://lh3.googleusercontent.com
https://avatars.githubusercontent.com;
font-src 'self' https://fonts.gstatic.com;
connect-src 'self' https://api.openai.com https://api-adresse.data.gouv.fr https://pub-*.r2.dev;
frame-src 'self' https://js.stripe.com https://hooks.stripe.com;
form-action 'self';
frame-ancestors 'none';
base-uri 'self';
```

CSP fixée après avoir bloqué initialement les factures R2, les avatars Google et l'autocomplete adresse (commit `549d6aa`).

## § 6.6 Rate limiting

Implémentation Redis-backed dans `src/lib/rate-limit.ts` (pattern `INCR + EXPIRE`).

| CATÉGORIE                  | LIMITE  | FENÊTRE |
|----------------------------|---------|---------|
| <code>/api/auth/*</code>   | 5 req   | 60 s    |
| <code>/api/admin/*</code>  | 50 req  | 60 s    |
| <code>/api/search</code>   | 30 req  | 60 s    |
| <code>/api/contact</code>  | 5 req   | 1 h     |
| <code>/api/chat</code>     | 5 req   | 60 s    |
| Autres <code>/api/*</code> | 100 req | 60 s    |

Headers de réponse :

- `X-RateLimit-Limit`
- `X-RateLimit-Remaining`
- `X-RateLimit-Reset` (timestamp Unix)

Réponse 429 avec `retryAfter` en secondes.

**Stratégie fail-open** : si Redis est injoignable, les requêtes passent (priorité disponibilité sur sécurité — choix justifié pour une plateforme commerce).

## § 6.7 Sécurisation des paiements

### 6.7.1 PCI-DSS

Les données de carte bancaire **ne touchent jamais** notre serveur — Stripe Checkout est hébergé chez Stripe (PCI-DSS Level 1 certifié). Notre serveur reçoit uniquement :

- `paymentIntentId` (référence Stripe)
- `last4`, `brand` (métadonnées non sensibles)

### 6.7.2 Vérification webhook

```
const sig = req.headers.get('stripe-signature');
let event: Stripe.Event;
try {
  event = getStripe().webhooks.constructEvent(body, sig, process.env.STRIPE_WEBHOOK_SECRET);
} catch (err) {
  return NextResponse.json({ error: 'Invalid signature' }, { status: 400 });
}
```

Empêche un attaquant de forger de fausses notifications de paiement (commit `e52ef85`).

### 6.7.3 Source unique de vérité

Les méthodes de paiement enregistrées sont stockées **uniquement** chez Stripe (via `stripeCustomerId`). Pas de duplication en base, pas de risque de désynchronisation, conformité PCI-DSS automatique (voir `docs/DECISIONS_ARCHITECTURALES_CDC.md`).

## § 6.8 RGPD

### 6.8.1 Droits implémentés

| DROIT         | ARTICLE | IMPLÉMENTATION                                                |
|---------------|---------|---------------------------------------------------------------|
| Information   | Art. 13 | Page <code>/legal/privacy</code> complète                     |
| Accès         | Art. 15 | <code>GET /api/profile/export</code> retourne JSON exhaustif  |
| Rectification | Art. 16 | Formulaires profil et adresses                                |
| Effacement    | Art. 17 | <code>DELETE /api/profile/delete</code> (anonymise commandes) |
| Portabilité   | Art. 20 | Export JSON structuré                                         |
| Opposition    | Art. 21 | Désactivation analytics, suppression compte                   |

### 6.8.2 Consentement cookies

- Bannière au premier accès (composant `CookieConsent`)
- Trois niveaux : essentiels (obligatoire), analytiques, marketing
- Choix persisté en `localStorage`
- Pas de tracking actif sans consentement explicite

### 6.8.3 Mesures techniques

- Mots de passe hashés `bcrypt`
- Logs anonymisés en production (pas d'IP/email en clair)
- Suppression cascade des données d'un compte effacé
- Anonymisation des Order après suppression du User (préservation comptable + RGPD)
- DPO contact : indiqué dans la politique de confidentialité

## § 6.9 Audits de sécurité

### 6.9.1 Audit interne automatisé

`npm run security:audit` exécute `scripts/security-audit.ts` qui produit `docs/security-report.md` (audit dépendances + headers + endpoints).

### 6.9.2 Audit GitHub

- **CodeQL** : analyse statique hebdomadaire (lundi 9h UTC) — workflow `.github/workflows/security.yml`
- **npm audit** : intégré au workflow CI

### 6.9.3 Patches sécurité notables

| DATE    | COMMIT               | CVE/ISSUE                                   | ACTION                                                                      |
|---------|----------------------|---------------------------------------------|-----------------------------------------------------------------------------|
| 2026-04 | <code>817a2d2</code> | XSS DOMPurify + xlsx CVE                    | Migration <code>xlsx</code> → <code>exceljs</code> , sanitization renforcée |
| 2026-03 | <code>e7c0443</code> | 35 vulnérabilités npm                       | <code>npm audit fix</code>                                                  |
| 2025-12 | <code>e52ef85</code> | Webhook Stripe non vérifié                  | Ajout signature <code>constructEvent</code>                                 |
| 2025-12 | <code>31171b6</code> | Endpoints users non protégés                | Ajout vérification ADMIN                                                    |
| 2025-12 | <code>248ae30</code> | POST <code>/api/products</code> non protégé | Ajout vérification ADMIN                                                    |
| 2025-12 | <code>6883c47</code> | <code>Console.log</code> en prod            | Suppression                                                                 |

### 6.9.4 Bug Redis Proxy en production

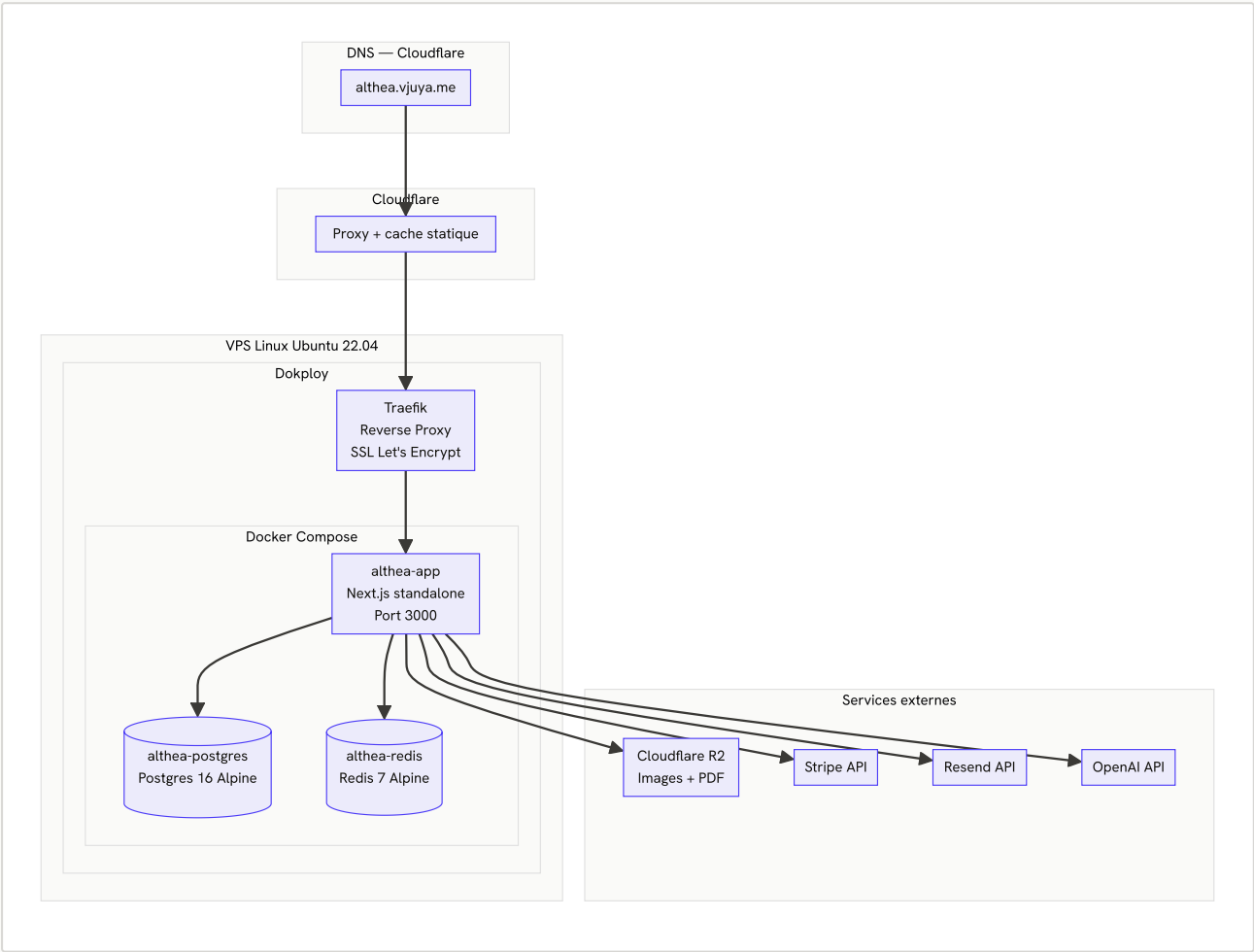
Le pattern Proxy initial pour l'init lazy de Redis ne préservait pas le contexte `this` d'ioredis, causant des erreurs 500 sur `/api/auth/session`. Remplacé par un objet wrapper explicite (commit `8381878`).

**Leçon retenue** : éviter les patterns JS avancés (Proxy, Reflect) avec des libs tierces sensibles au binding.



# 7. Déploiement

## § 7.1 Architecture de déploiement



| COMPOSANT | SPÉCIFICATIONS                 |
|-----------|--------------------------------|
| OS        | Ubuntu 22.04 LTS               |
| RAM       | 2 Go minimum (4 Go recommandé) |
| CPU       | 2 vCPU                         |
| Disque    | 20 Go SSD                      |
| Docker    | 24+                            |
| Compose   | v2+                            |
| Domaine   | althea.vjuya.me                |



| COMPOSANT | SPÉCIFICATIONS                      |
|-----------|-------------------------------------|
| TLS       | Let's Encrypt (renouvellement auto) |

## § 7.2 Conteneurisation Docker

### 7.2.1 Dockerfile multi-stage

Trois stages pour optimiser taille et perfs :

```
# Stage 1 – deps : installation des dépendances
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

# Stage 2 – builder : génération Prisma + build Next.js
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN --mount=type=cache,target=/app/.next/cache npm run build

# Stage 3 – runner : image de production minimale
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000
ENV HOSTNAME=0.0.0.0
RUN addgroup --system --gid 1001 nodejs && adduser --system --uid 1001 nextjs
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static
COPY --from=builder --chown=nextjs:nodejs /app/public ./public
USER nextjs
EXPOSE 3000
CMD ["node", "server.js"]
```

#### Métriques :

- Image finale : **432 Mo** (vs 3,16 Go avec Nixpacks → -86 %)
- Build cold : **5 min 23 s** (vs 9 min 20 s)
- Build cached (BuildKit) : **2-3 min**

### 7.2.2 docker-compose.yml (local)

Trois services + 3 outils dev en profil `dev` :

| SERVICE          | IMAGE       | PORT | VOLUME | HEALTHCHECK                   |
|------------------|-------------|------|--------|-------------------------------|
| <code>app</code> | Build local | 3000 | —      | HTTP <code>/api/health</code> |

| SERVICE                      | IMAGE                                                                            | PORT | VOLUME        | HEALTHCHECK    |
|------------------------------|----------------------------------------------------------------------------------|------|---------------|----------------|
| postgres                     | postgres:16-alpine                                                               | 5432 | postgres_data | pg_isready     |
| redis                        | redis:7-alpine --appendonly yes --maxmemory 256mb --maxmemory-policy allkeys-lru | 6379 | redis_data    | redis-cli ping |
| adminer (profil dev)         | adminer                                                                          | 8080 | —             | —              |
| redis-commander (profil dev) | rediscommander/redis-commander                                                   | 8081 | —             | —              |
| mailhog (profil dev)         | mailhog/mailhog                                                                  | 8025 | —             | —              |

Réseau bridge `althea-network` partagé.

## § 7.3 Pipeline CI/CD

### 7.3.1 GitHub Actions ( `.github/workflows/ci.yml` )

| JOB             | ACTION                                            | DÉPEND DE       |
|-----------------|---------------------------------------------------|-----------------|
| lint            | <code>npm run lint</code> (ESLint v9)             | —               |
| typecheck       | <code>npx tsc --noEmit</code>                     | —               |
| build           | <code>npm run build</code> (avec Prisma generate) | lint, typecheck |
| prisma-validate | <code>npx prisma validate</code>                  | —               |

Variables d'environnement placeholder pour CI (commit `163362f`) :

```
env:
  NEXTAUTH_SECRET: ${ secrets.NEXTAUTH_SECRET || 'ci-build-secret-placeholder' }
  NEXTAUTH_URL: http://localhost:3000
  DATABASE_URL: ${ secrets.DATABASE_URL ||
'postgresql://placeholder:placeholder@localhost:5432/placeholder' }
```

**Concurrency group** : annule les builds obsolètes sur push successifs.

### 7.3.2 Workflow security

- `npm audit` avec seuil critical/high
- CodeQL analyse hebdomadaire (cron lundi 9h UTC)

### 7.3.3 Workflow Docker

Build automatique de l'image Docker à chaque PR pour valider le Dockerfile.

### 7.3.4 Auto-deploy via Dokploy

- Push sur `main` → Dokploy détecte → rebuild Docker → redéploiement zero-downtime
- Logs disponibles dans le dashboard Dokploy
- Rollback en 1 clic vers déploiement précédent

## § 7.4 Variables d'environnement

### 7.4.1 Application

| VARIABLE                          | EXEMPLE PROD                         | DESCRIPTION  |
|-----------------------------------|--------------------------------------|--------------|
| <code>NODE_ENV</code>             | <code>production</code>              | —            |
| <code>NEXT_PUBLIC_APP_URL</code>  | <code>https://althea.vjuya.me</code> | URL publique |
| <code>NEXT_PUBLIC_APP_NAME</code> | <code>Althea Systems</code>          | Nom affiché  |

### 7.4.2 Base de données

| VARIABLE                  | DESCRIPTION                                                       |
|---------------------------|-------------------------------------------------------------------|
| <code>DATABASE_URL</code> | <code>postgresql://althea:PASSWORD@postgres:5432/althea_db</code> |

### 7.4.3 Authentification

| VARIABLE                     | DESCRIPTION                                     |
|------------------------------|-------------------------------------------------|
| <code>NEXTAUTH_SECRET</code> | Généré par <code>openssl rand -base64 32</code> |
| <code>NEXTAUTH_URL</code>    | URL publique (utilisée pour redirects email)    |

### 7.4.4 OAuth

| VARIABLE                          | SOURCE               |
|-----------------------------------|----------------------|
| <code>GOOGLE_CLIENT_ID</code>     | Google Cloud Console |
| <code>GOOGLE_CLIENT_SECRET</code> | Google Cloud Console |
| <code>GITHUB_CLIENT_ID</code>     | GitHub OAuth App     |

| VARIABLE             | SOURCE           |
|----------------------|------------------|
| GITHUB_CLIENT_SECRET | GitHub OAuth App |

Callbacks à configurer : <https://althea.vjuya.me/api/auth/callback/{provider}> .

### 7.4.5 Stripe

| VARIABLE                           | DESCRIPTION                             |
|------------------------------------|-----------------------------------------|
| STRIPE_SECRET_KEY                  | sk_live_...                             |
| NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY | pk_live_...                             |
| STRIPE_WEBHOOK_SECRET              | whsec_... (généré sur Stripe Dashboard) |

Webhook URL à configurer Stripe : <https://althea.vjuya.me/api/stripe/webhook> .

### 7.4.6 Redis

| VARIABLE  | DESCRIPTION        |
|-----------|--------------------|
| REDIS_URL | redis://redis:6379 |

### 7.4.7 Cloudflare R2

| VARIABLE             | DESCRIPTION                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|
| R2_ACCOUNT_ID        | Cloudflare Dashboard                                                                                                  |
| R2_ACCESS_KEY_ID     | R2 API Token                                                                                                          |
| R2_SECRET_ACCESS_KEY | R2 API Token                                                                                                          |
| R2_BUCKET_NAME       | althea-images                                                                                                         |
| R2_PUBLIC_URL        | <a href="https://pub-578e3fbcadaa433fb571ec293b300e3c.r2.dev">https://pub-578e3fbcadaa433fb571ec293b300e3c.r2.dev</a> |

### 7.4.8 Resend

| VARIABLE          | DESCRIPTION                               |
|-------------------|-------------------------------------------|
| RESEND_API_KEY    | re_...                                    |
| RESEND_FROM_EMAIL | noreply@vjuya.me (domaine vérifié Resend) |

### 7.4.9 OpenAI (chatbot)

| VARIABLE                    | DESCRIPTION         |
|-----------------------------|---------------------|
| <code>OPENAI_API_KEY</code> | <code>sk-...</code> |

### 7.4.10 Logging

| VARIABLE               | DESCRIPTION                                          |
|------------------------|------------------------------------------------------|
| <code>LOG_LEVEL</code> | <code>warn</code> en prod, <code>debug</code> en dev |

## § 7.5 Procédure de déploiement

### 7.5.1 Déploiement initial (one-time)

```
# 1. Provisionner VPS Ubuntu 22.04
# 2. Installer Dokploy
curl -sSL https://dokploy.com/install.sh | sh

# 3. Accéder au dashboard
# https://<ip-vps>:3000

# 4. Créer une application Dokploy
#   - Source: GitHub repo itsaam/althea-systems
#   - Branche: main
#   - Build pack: Dockerfile (racine)
#   - Domaine: althea.vjuya.me
#   - SSL: Let's Encrypt

# 5. Déployer Postgres et Redis (services Dokploy intégrés)

# 6. Configurer toutes les variables d'environnement (voir §7.4)

# 7. Trigger initial deploy
```

### 7.5.2 Migrations base de données

Au premier déploiement et à chaque migration :

```
docker exec althea-app npx prisma migrate deploy
docker exec althea-app npx prisma db seed # une seule fois
```

### 7.5.3 Mises à jour ultérieures

Push sur `main` → Dokploy rebuild + redéploie automatiquement. Les migrations Prisma s'exécutent au start du container ( `npm run start` = `prisma migrate deploy && next start` ).

## § 7.6 Rollback

### 7.6.1 Rollback application

Via Dokploy : un clic « Redeploy » sur la version précédente. Temps : < 5 min.

Manuel :

```
docker compose -f docker/docker-compose.yml down
git checkout <commit-stable>
docker compose -f docker/docker-compose.yml up -d --build
```

### 7.6.2 Rollback base de données

```
# Marque migration comme rollbackée
docker exec althea-app npx prisma migrate resolve --rolled-back <name>

# Restaure backup si nécessaire
docker exec -i althea-postgres pg_restore -U althea -d althea_db --clean < backup.dump
```

### 7.6.3 Procédure d'urgence

1. **Détection** : alerte UptimeRobot ou Sentry (à venir)
2. **Évaluation** : < 5 min — consulter logs Dokploy + Winston
3. **Action** : redeploy version précédente Dokploy
4. **Post-mortem** : document dans `docs/reports/`

## § 7.7 Backups

### 7.7.1 Base de données

Script `/opt/scripts/backup-althea.sh` :

```
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M)
BACKUP_DIR=/opt/backups/althea
mkdir -p $BACKUP_DIR
docker exec althea-postgres pg_dump -U althea althea_db -F c > $BACKUP_DIR/althea_${DATE}.dump
ls -t $BACKUP_DIR/*.dump | tail -n +31 | xargs rm -f 2>/dev/null
```

Cron : `0 3 * * * /opt/scripts/backup-althea.sh >> /var/log/althea-backup.log 2>&1`

| PARAMÈTRE | VALEUR             |
|-----------|--------------------|
| Fréquence | Quotidienne 3h UTC |

| PARAMÈTRE | VALEUR                                        |
|-----------|-----------------------------------------------|
| Format    | <code>pg_dump -F c</code> (logique compressé) |
| Rétention | 30 jours quotidiens, 12 mois mensuels         |
| Stockage  | <code>/opt/backups/althea/</code> sur le VPS  |

### 7.7.2 Restauration

```
docker exec -i althea-postgres pg_restore -U althea -d althea_db --clean < /opt/backups/althea/althea_YYYYMMDD.dump
```

### 7.7.3 Stockage R2

Versioning **désactivé** (économie d'espace). Backup R2 envisagé via cron `rcclone` vers stockage tiers (roadmap).

## § 7.8 Monitoring

| OUTIL                | USAGE                                                                               |
|----------------------|-------------------------------------------------------------------------------------|
| UptimeRobot          | Surveillance HTTP toutes les 5 min sur <code>/</code> et <code>/api/products</code> |
| Dokploy dashboard    | Logs, métriques container, redéploiement                                            |
| Winston logs         | Logs applicatifs accessibles via <code>docker logs althea-app</code>                |
| Stripe Dashboard     | Monitoring des paiements et webhooks                                                |
| Cloudflare Analytics | Trafic, bande passante, attaques bloquées                                           |

### SLA cibles

| MÉTRIQUE    | CIBLE                          | MESURE                      |
|-------------|--------------------------------|-----------------------------|
| Uptime      | 99,9 % (8,7 h max downtime/an) | UptimeRobot                 |
| Réponse API | < 200 ms (P95)                 | Logs Winston                |
| LCP         | < 2 s                          | Lighthouse, Core Web Vitals |
| RTO         | < 30 min                       | Procédure rollback          |
| RPO         | < 24 h                         | Backup quotidien            |

## § 7.9 Versionning

### 7.9.1 Workflow Git

- Branche principale : `main` (protégée)
- Branches feature : `feat/*`, `fix/*`, `chore/*`, `docs/*`, `refactor/*`
- Pull Requests obligatoires sur `main`
- CI vert requis avant merge
- Auto-merge activable si CI verte (rare, préférence revue manuelle)

### 7.9.2 Conventional Commits

Format imposé : `type(scope): description`

| TYPE                  | USAGE                                  |
|-----------------------|----------------------------------------|
| <code>feat</code>     | Nouvelle fonctionnalité                |
| <code>fix</code>      | Correction de bug                      |
| <code>chore</code>    | Maintenance, dépendances               |
| <code>docs</code>     | Documentation seule                    |
| <code>refactor</code> | Refonte sans changement fonctionnel    |
| <code>test</code>     | Ajout/modification de tests            |
| <code>perf</code>     | Optimisation performance               |
| <code>style</code>    | Mise en forme code (espace, formatage) |
| <code>ci</code>       | Pipeline CI/CD                         |

Exemples (vrais commits du projet) :

- `feat(seo): audit SEO complet – brand ranking 'Althea Systems'`
- `fix(auth): redirection verify-email utilise NEXTAUTH_URL au lieu de request.url`
- `refactor(pdf+admin): facture PDF brandée Althea + users badges neutres`
- `chore(deps): patch 35 vulnérabilités via npm audit fix`

### 7.9.3 Tags et releases

Tags `v0.x.y` au format SemVer pour les jalons importants. Releases GitHub avec changelog auto-généré.



### 7.9.4 Repo

- **GitHub** : `https://github.com/itsaam/althea-systems`
  - **Visibilité** : privé (projet pédagogique)
  - **Branches protégées** : `main` (require PR + CI green)
  - **Labels PR auto** : workflow `label-pr.yml` étiquette selon les fichiers modifiés (`frontend`, `backend`, `database`, `ci`, etc.)
-



# 8. Annexes

## § 8.1 Catalogue exhaustif des 72 endpoints API

### 8.1.1 Authentification ( /api/auth )

| MÉTHODE  | ENDPOINT                  | DESCRIPTION                             | AUTH   |
|----------|---------------------------|-----------------------------------------|--------|
| ALL      | /api/auth/[...nextauth]   | Handler NextAuth (login/logout/session) | Public |
| POST     | /api/auth/register        | Inscription                             | Public |
| GET/POST | /api/auth/verify-email    | Vérification email                      | Public |
| POST     | /api/auth/forgot-password | Demande reset MDP                       | Public |
| POST     | /api/auth/reset-password  | Reset MDP avec token                    | Public |
| POST     | /api/auth/2fa/setup       | Init 2FA (génère QR)                    | Auth   |
| POST     | /api/auth/2fa/verify      | Vérifie code TOTP                       | Auth   |

### 8.1.2 Catalogue ( /api/products , /api/categories )

| MÉTHODE | ENDPOINT                   | DESCRIPTION                    | AUTH   |
|---------|----------------------------|--------------------------------|--------|
| GET     | /api/products              | Liste paginée, filtrable       | Public |
| POST    | /api/products              | Créer produit                  | ADMIN  |
| GET     | /api/products/[id]         | Détail produit                 | Public |
| GET     | /api/products/[id]/similar | Produits similaires            | Public |
| POST    | /api/products/[id]/images  | Upload image produit           | ADMIN  |
| GET     | /api/products/featured     | Produits mis en avant          | Public |
| GET     | /api/products/search       | Recherche full-text            | Public |
| GET     | /api/categories            | Liste catégories (cache 30min) | Public |
| POST    | /api/categories            | Créer catégorie                | ADMIN  |
| GET     | /api/categories/[id]       | Détail catégorie               | Public |
| PUT     | /api/categories/[id]       | Modifier catégorie             | ADMIN  |

| MÉTHODE | ENDPOINT                             | DESCRIPTION       | AUTH   |
|---------|--------------------------------------|-------------------|--------|
| PUT     | <code>/api/categories/reorder</code> | Réordonner        | ADMIN  |
| GET     | <code>/api/search</code>             | Recherche globale | Public |

### 8.1.3 Commandes et paiements

| MÉTHODE | ENDPOINT                                    | DESCRIPTION             | AUTH        |
|---------|---------------------------------------------|-------------------------|-------------|
| GET     | <code>/api/orders</code>                    | Mes commandes           | Auth        |
| POST    | <code>/api/orders</code>                    | Créer commande          | Auth        |
| GET     | <code>/api/orders/[id]</code>               | Détail commande         | Owner/ADMIN |
| PATCH   | <code>/api/orders/[id]</code>               | Changer statut          | ADMIN       |
| GET     | <code>/api/orders/[id]/invoice</code>       | Facture PDF             | Owner/ADMIN |
| GET     | <code>/api/cart</code>                      | Récupérer panier        | Auth        |
| POST    | <code>/api/cart</code>                      | Ajouter au panier       | Auth        |
| PUT     | <code>/api/cart</code>                      | Modifier quantité       | Auth        |
| POST    | <code>/api/stripe/checkout</code>           | Créer session Stripe    | Auth        |
| POST    | <code>/api/stripe/payment-saved-card</code> | Payer carte enregistrée | Auth        |
| POST    | <code>/api/stripe/webhook</code>            | Webhook Stripe          | Stripe sig  |

### 8.1.4 Comptes et utilisateurs

| MÉTHODE | ENDPOINT                                                | DESCRIPTION           | AUTH  |
|---------|---------------------------------------------------------|-----------------------|-------|
| GET     | <code>/api/users</code>                                 | Liste utilisateurs    | ADMIN |
| POST    | <code>/api/users</code>                                 | Créer utilisateur     | ADMIN |
| GET     | <code>/api/users/[id]</code>                            | Détail utilisateur    | ADMIN |
| PATCH   | <code>/api/users/[id]</code>                            | Modifier utilisateur  | ADMIN |
| DELETE  | <code>/api/users/[id]</code>                            | Supprimer utilisateur | ADMIN |
| GET     | <code>/api/users/me</code>                              | Mon profil            | Auth  |
| POST    | <code>/api/users/me/payment-methods</code>              | Ajouter carte         | Auth  |
| DELETE  | <code>/api/users/me/payment-methods/[id]</code>         | Supprimer carte       | Auth  |
| PUT     | <code>/api/users/me/payment-methods/[id]/default</code> | Définir par défaut    | Auth  |

| MÉTHODE | ENDPOINT                        | DESCRIPTION        | AUTH  |
|---------|---------------------------------|--------------------|-------|
| GET     | /api/profile                    | Profil détaillé    | Auth  |
| PUT     | /api/profile                    | Modifier profil    | Auth  |
| GET     | /api/profile/export             | Export RGPD        | Auth  |
| DELETE  | /api/profile/delete             | Supprimer compte   | Auth  |
| GET     | /api/addresses                  | Mes adresses       | Auth  |
| POST    | /api/addresses                  | Ajouter adresse    | Auth  |
| GET     | /api/addresses/[id]             | Détail adresse     | Owner |
| PATCH   | /api/addresses/[id]             | Modifier adresse   | Owner |
| DELETE  | /api/addresses/[id]             | Supprimer adresse  | Owner |
| POST    | /api/addresses/[id]/set-default | Définir par défaut | Owner |

### 8.1.5 Factures, avoirs, contenus

| MÉTHODE | ENDPOINT                   | DESCRIPTION          | AUTH              |
|---------|----------------------------|----------------------|-------------------|
| GET     | /api/invoices              | Liste factures       | Auth              |
| GET     | /api/invoices/[id]         | Détail facture       | Owner/ADMIN       |
| GET     | /api/invoices/[id]/pdf     | PDF facture          | Owner/ADMIN       |
| GET     | /api/credits               | Liste avoirs         | ADMIN             |
| GET     | /api/credits/[id]          | Détail avoir         | ADMIN             |
| GET     | /api/credits/[id]/download | PDF avoir            | ADMIN             |
| PATCH   | /api/credits               | Créer/modifier avoir | ADMIN             |
| DELETE  | /api/credits               | Supprimer avoir      | ADMIN             |
| GET     | /api/carousel              | Slides carrousel     | Public            |
| POST    | /api/carousel              | Créer slide          | ADMIN             |
| GET     | /api/carousel/[id]         | Détail slide         | Public            |
| PUT     | /api/carousel/[id]         | Modifier slide       | ADMIN             |
| DELETE  | /api/carousel/[id]         | Supprimer slide      | ADMIN             |
| GET     | /api/contact               | Liste messages       | ADMIN             |
| POST    | /api/contact               | Envoyer message      | Public (rate-lim) |

| MÉTHODE | ENDPOINT                 | DESCRIPTION | AUTH  |
|---------|--------------------------|-------------|-------|
| PUT     | /api/contact/[id]/status | Marquer lu  | ADMIN |

8.1.6 Upload et chat

| MÉTHODE | ENDPOINT           | DESCRIPTION        | AUTH              |
|---------|--------------------|--------------------|-------------------|
| POST    | /api/upload        | Upload R2          | ADMIN             |
| POST    | /api/images/upload | Upload image       | ADMIN             |
| GET     | /api/images/[key]  | Proxy image (CORS) | Public            |
| POST    | /api/chat          | Streaming chatbot  | Public (rate-lim) |

8.1.7 Statistiques

| MÉTHODE | ENDPOINT             | DESCRIPTION     | AUTH  |
|---------|----------------------|-----------------|-------|
| GET     | /api/stats           | Stats générales | ADMIN |
| GET     | /api/stats/revenue   | Stats CA        | ADMIN |
| GET     | /api/stats/customers | Stats clients   | ADMIN |

8.1.8 Admin ( /api/admin/\* )

| MÉTHODE | ENDPOINT                            | DESCRIPTION          | AUTH  |
|---------|-------------------------------------|----------------------|-------|
| GET     | /api/admin/dashboard/kpis           | KPIs dashboard       | ADMIN |
| GET     | /api/admin/dashboard/sales-chart    | Graph ventes         | ADMIN |
| GET     | /api/admin/dashboard/category-sales | Ventes par cat       | ADMIN |
| GET     | /api/admin/dashboard/cart-analysis  | Analyse paniers      | ADMIN |
| GET     | /api/admin/products                 | Liste admin produits | ADMIN |
| GET     | /api/admin/products/[id]            | Détail admin produit | ADMIN |
| PUT     | /api/admin/products/[id]            | MAJ produit          | ADMIN |
| DELETE  | /api/admin/products/[id]            | Supprimer produit    | ADMIN |
| PATCH   | /api/admin/products/bulk            | Action en masse      | ADMIN |
| DELETE  | /api/admin/products/bulk            | Suppression en masse | ADMIN |
| GET     | /api/admin/products/export          | Export Excel         | ADMIN |

| MÉTHODE | ENDPOINT                              | DESCRIPTION         | AUTH  |
|---------|---------------------------------------|---------------------|-------|
| POST    | /api/admin/products/reorder-featured  | Réordonner featured | ADMIN |
| GET     | /api/admin/orders/export              | Export commandes    | ADMIN |
| GET     | /api/admin/users/export               | Export users        | ADMIN |
| GET     | /api/admin/invoices/export            | Export factures     | ADMIN |
| GET     | /api/admin/credits/export             | Export avoirs       | ADMIN |
| GET     | /api/admin/messages/export            | Export messages     | ADMIN |
| POST    | /api/admin/messages/[id]/reply        | Répondre message    | ADMIN |
| GET     | /api/admin/stock/alerts               | Alertes stock       | ADMIN |
| GET     | /api/admin/chatbot/conversations      | Liste conversations | ADMIN |
| GET     | /api/admin/chatbot/conversations/[id] | Détail conversation | ADMIN |

### 8.1.9 Documentation API

| MÉTHODE | ENDPOINT  | DESCRIPTION  |
|---------|-----------|--------------|
| GET     | /api/docs | OpenAPI JSON |
| GET     | /docs     | Swagger UI   |

## § 8.2 Catalogue des composants principaux

### Layout

- Header, Footer, MobileMenu, Navigation
- LanguageSwitcher, ThemeToggle

### Home

- HeroEditorial, BlueprintSvg, SplitText, MagneticButton
- CtaProductsCarousel (GSAP), AdminCarousel (Embla)
- MarksTicker, CategoriesGrid, FeaturedProducts

### Products

- ProductCard, ProductGallery, ProductDetail
- FilterBar, SortDropdown, PriceRangeSlider

## Cart & Checkout

- `CartItem`, `CartSummary`, `CartDrawer`
- `CheckoutForm`, `AddressForm`, `OrderSummary`

## Auth

- `LoginForm`, `RegisterForm`, `ForgotPasswordForm`, `ResetPasswordForm`
- `Verify2faForm`, `Setup2faQr`, `BackupCodesDisplay`

## Admin

- `AdminShell`, `AdminSidebar`, `AdminHeader`
- `DataTable` (`TanStack`), `BulkActions`, `ExportButton`
- `ProductForm`, `CategoryTree`, `OrderTimeline`
- `KpiCard`, `SalesChart`, `CategoryPie`

## Shared

- `Breadcrumb`, `Pagination`, `EmptyState`
- `LoadingSkeleton`, `ErrorBoundary`

## Chatbot

- `ChatBubble`, `ChatWindow`, `MessageList`, `MessageInput`

## SEO

- `JsonLd`, `OgImage`, `BreadcrumbSeo`

## TVA

- `PriceDisplay` (HT/TTC/breakdown)

## UI (shadcn)

- `Button`, `Input`, `Label`, `Dialog`, `Sheet`, `Dropdown`, `Select`, `Tabs`, `Switch`, `Checkbox`, `Avatar`, `Separator`, `Slider`

## § 8.3 Variables d'environnement complètes ( `.env.example` )

```
# === APPLICATION ===  
NODE_ENV=development  
NEXT_PUBLIC_APP_NAME=Althea Systems
```



```
NEXT_PUBLIC_APP_URL=http://localhost:3000

# === DATABASE ===
DATABASE_URL="postgres://althea:password@localhost:5432/althea_db"

# === REDIS ===
REDIS_URL="redis://localhost:6379"

# === AUTH ===
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=change-me-with-openssl-rand-base64-32

# === OAUTH ===
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=

# === STRIPE ===
STRIPE_SECRET_KEY=sk_test...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test...
STRIPE_WEBHOOK_SECRET=whsec...

# === RESEND ===
RESEND_API_KEY=re...
RESEND_FROM_EMAIL=onboarding@resend.dev

# === CLOUDFLARE R2 ===
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET_NAME=althea-images
R2_PUBLIC_URL=https://pub-xxx.r2.dev

# === OPENAI ===
OPENAI_API_KEY=sk-...

# === LOGGING ===
LOG_LEVEL=info
LOG_DIR=logs
```

## § 8.4 Glossaire

| TERME             | DÉFINITION                                                                                                                                                               |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>App Router</b> | Router Next.js basé sur le dossier <code>app/</code> (par opposition au Pages Router historique). Supporte les Server Components, le streaming et les layouts imbriqués. |
| <b>B2B</b>        | Business-to-Business — vente entre professionnels.                                                                                                                       |
| <b>B2C</b>        | Business-to-Consumer — vente au particulier.                                                                                                                             |
| <b>CDN</b>        | Content Delivery Network — réseau distribué qui sert les assets statiques depuis le serveur le plus proche.                                                              |
| <b>CSP</b>        | Content Security Policy — header HTTP qui restreint les sources autorisées (scripts, images, frames).                                                                    |
| <b>CUID</b>       | Collision-Resistant Unique Identifier — type d'ID utilisé par Prisma.                                                                                                    |

| TERME               | DÉFINITION                                                                                                                         |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Edge Runtime</b> | Runtime JavaScript léger (V8 isolate) exécuté en bordure réseau. Limité par rapport à Node.js (pas de fs, etc.) mais ultra-rapide. |
| <b>HSTS</b>         | HTTP Strict Transport Security — force le HTTPS pour une durée donnée.                                                             |
| <b>JWT</b>          | JSON Web Token — format de token signé permettant la stateless authentication.                                                     |
| <b>LCP</b>          | Largest Contentful Paint — métrique Core Web Vitals : temps d'apparition du plus gros élément visible.                             |
| <b>MFA / 2FA</b>    | Multi-Factor Authentication — exige un second facteur (TOTP, SMS) en plus du mot de passe.                                         |
| <b>OAuth</b>        | Standard d'autorisation déléguée (Google, GitHub agissent comme fournisseurs d'identité).                                          |
| <b>PCI-DSS</b>      | Payment Card Industry Data Security Standard — référentiel de sécurité pour le traitement des cartes bancaires.                    |
| <b>RGPD</b>         | Règlement Général sur la Protection des Données — réglementation européenne sur les données personnelles.                          |
| <b>RPO</b>          | Recovery Point Objective — perte de données acceptable en cas d'incident (24 h ici = backup quotidien).                            |
| <b>RSC</b>          | React Server Components — composants React rendus uniquement côté serveur, sans envoi de JavaScript au client.                     |
| <b>RTO</b>          | Recovery Time Objective — temps maximum acceptable pour rétablir le service.                                                       |
| <b>SaaS</b>         | Software as a Service — logiciel hébergé chez un tiers (Stripe, Resend, OpenAI).                                                   |
| <b>Slug</b>         | Identifiant URL-friendly (ex : <code>stethoscope-cardio-pro</code> plutôt qu'un ID numérique).                                     |
| <b>SSR</b>          | Server-Side Rendering — rendu HTML par le serveur à chaque requête.                                                                |
| <b>TOTP</b>         | Time-based One-Time Password (RFC 6238) — code à usage unique généré toutes les 30 secondes.                                       |
| <b>TTL</b>          | Time To Live — durée de validité d'une donnée (cache, token).                                                                      |
| <b>VPS</b>          | Virtual Private Server — serveur virtualisé loué chez un hébergeur.                                                                |
| <b>WAF</b>          | Web Application Firewall — pare-feu applicatif (Cloudflare en intègre un).                                                         |
| <b>WYSIWYG</b>      | What You See Is What You Get — éditeur visuel (Tiptap ici).                                                                        |
| <b>Webhook</b>      | Notification HTTP envoyée par un service tiers à notre serveur (ex : Stripe sur paiement réussi).                                  |

## § 8.5 Ressources et références

### Documentation officielle utilisée

- [Next.js 16](#)
- [React 19](#)

- [Prisma 6](#)
- [NextAuth.js v4](#)
- [Tailwind CSS 4](#)
- [Stripe API](#)
- [WCAG 2.1](#)
- [OWASP Top 10](#)

## Outils

- [Dokploy](#)
- [Cloudflare R2](#)
- [Resend](#)
- [Mermaid](#)

## Standards

- RGPD — Règlement (UE) 2016/679
- PCI-DSS v4.0
- RFC 6238 (TOTP)
- RFC 7519 (JWT)

## Documents internes du projet

- `docs/DOCUMENT_CADRAGE.pdf` — Document de cadrage (livrable VI.3)
- `docs/RAPPORT_SOUTENANCE_SAMY.md` — Rapport technique soutenance
- `docs/A11Y_AUDIT.md` — Audit accessibilité WCAG 2.1 AA
- `docs/security-report.md` — Rapport audit sécurité
- `docs/RESPONSIVE_AUDIT.md` — Audit responsive mobile-first
- `docs/DECISIONS_ARCHITECTURALES_CDC.md` — Décisions architecturales motivées
- `README.md` — Point d'entrée projet
- `CONTRIBUTING.md` — Guide contribution

---

## Fin du Dossier de Conception Technique — Althea Systems v2.0

*Document généré le 7 mai 2026 — 8 sections, 8 annexes, ~80 pages.*